Seminar 1 - Exerciții rezolvate

Echipa AA

11 noiembrie 2022

Cuprins

Exercițiul 1 **1**
Rezolvare . . . 1
Generalizare . . . 2

Exercițiul 2 **3**
Rezolvare . . . 3

Exercițiul 3 **3**
Rezolvare . . . 3

Exercițiul 4 **3**
Rezolvare . . . 3

Exercițiul 5 **4**
Rezolvare . . . 4

Exercițiul 6 **4**
Rezolvare . . . 4

Exercițiul 7 **5**
Rezolvare . . . 5

Exercițiul 8 **6**
Rezolvare . . . 6
Caz particular cunoscut . . . 6
Observație . . . 6

Bibliografie **6**

Exercițiul 1

Fie mulțimile A, B disjuncte și complementare. Demonstrați că dacă A, B sunt recursiv-enumerabile, atunci acestea sunt și recursive.

Rezolvare

Rescriem ipoteza în termeni matematici:

1. $A \cap B = \emptyset$
2. $A \cup B = \mathbb{N}$
3. $A, B \in \mathcal{RE}$

Notăm cu $\mathcal{R}$ mulțimea tuturor mulțimilor recursive, cu $\mathcal{RE}$ mulțimea tuturor mulțimilor recursive-enumerabile, iar $\mathcal{P}$ cu este mulțimea tuturor programelor.

Din 3 rezultă că $\exists P_A, P_B \in \mathcal{P}$, astfel încât

$$P_A(x) = \begin{cases} 1 & \text{dacă } x \in A \\ \perp & \text{altfel} \end{cases}$$

$$P_B(x) = \begin{cases} 1 & \text{dacă } x \in B \\ \bot & \text{altfel} \end{cases}$$

Pentru a demonstra că $A \in \mathcal{R}$ trebuie să găsim $Q_A \in \mathcal{P}$, astfel încât

$$Q_A(x) = \begin{cases} 1 & \text{dacă } x \in A \\ 0 & \text{altfel} \end{cases}$$

Pentru a îl construi pe Q_A ne vom folosi de ce programe avem deja, P_A și P_B, și în funcție de outputurile lor să ne dăm seama dacă putem afirma/infirma apartenența la A.

A returnat P_A 1 pentru x?	A returnat P_B 1 pentru x?	$x \in A$?
DA	NU	DA
NU	DA	NU

Observați că nu am analizat cazul în care ambele programe returnează același rezultat. Acest fapt este imposibil, deoarece putem deduce că $\forall x \in \mathbb{N}, x \in A$ sau (exclusiv) $x \in B$, fie din desen, fie din ipotezele 1 și 2.

Pentru a putea scrie programul Q_A și nu știm care din P_A sau P_B se termină. Putem simula o paralelizare a celor două rulând un număr finit de instrucțiuni din primul program, apoi ruleazând un număr finit de instrucțiuni din al doilea, și tot așa. (Acest procedeu se cheamă în engleză dovetailing. Există o asemănare cu noțiunea de multitasking.)

```
// rulează funcția P asupra inputului x în steps pași
// dacă funcția a returnat în mai puțini pași, returnează valoarea ei
// altfel returnează o referință nulă
Boolean run_in_steps(BooleanFunction P, Input x, Integer steps);

bool QA(int x) {
  for (Integer t = 0; ; t++) { // t crește la infinit
    if (run_in_steps(PA, x, t) == true)
      return true;

    if (run_in_steps(PB, x, t) == true)
      return false;
  }
}

bool QB(int x) {
  return !QA(x);
}
```

Deoarce am găsit un program, atât pentru A, cât și pentru B, care răspunde într-un timp finit da sau nu legat de apartenența unui input la o mulțime, **ambele mulțimi sunt recursive**.

Generalizare

Fie $A_1, A_2, \ldots, A_n \in \mathcal{RE}$.

1. $\bigcap_{i=1}^{n} A_i = \emptyset$
2. $\bigcup_{i=1}^{n} A_i = \mathbb{N}$

$$P_A(i, x) = \begin{cases} 1 & \text{dacă } x \in A_i \\ \bot & \text{altfel} \end{cases}$$

```
bool Q_A(int i, int x) {
  for (Integer t = 0; ; t++) {
    if (run_in_steps(PA, (i, x), t) == true)
      return true;

    for (int j = 1; j <= n; j++)
      if (i != j && run_in_steps(PA, (j, x), t) == true)
        return false;
  }
}
```

Exercițiul 2

Fie $A \in \mathcal{R}$ și $B \in \mathcal{RE} \setminus \mathcal{R}$. Este $A \cup B \in \mathcal{RE}$?

Rezolvare

Fie P_A, P_B programele asociate mulțimilor A, B.

Știm că P_A se termină mereu, iar P_B se termină măcar doar când inputul aparține lui B. Construim $Q \in \mathcal{P}$ care poate afirma apartenența la $A \cup B$,

```
bool Q(int x) {
  if (PA(x) == true)
    return true;
  return PB(x);
}
```

Exercițiul 3

Pornind de la exercițiul anterior, demonstrați că $A \cup B \in \mathcal{R}$ sau $A \cup B \notin \mathcal{R}$.

Rezolvare

1. $A = \emptyset$ și $B = \{(P, x) \in \mathcal{P} \times \mathbb{N} | P(x) \neq \perp\}$, adică:
    1. A este mulțimea vidă
    2. B este mulțimea tuplurilor de programe și inputuri, cu propietatea că acel program se oprește pe acel input
    3. $A \cup B = B, B \in \mathcal{RE} \setminus \mathcal{R}$[1]
2. $A = \mathcal{P} \times \mathbb{N}$ și $B = \{(P, x) \in \mathcal{P} \times \mathbb{N} | P(x) \neq \perp\}$, unde:
    1. A este mulțimea tuturor programelor și a inputurilor
    2. Deoarce verificarea că un șir este un program sau nu e o problemă decidabilă (spre exemplu compilatoarele acceptă sau nu un program), mulțimea A este recursivă
    3. $A \cup B = A, A \in \mathcal{R}$

Exercițiul 4

Mulțimea $A \in \mathcal{RE} \setminus \mathcal{R}$. Ce putem spune despre complementul lui A, $A^{\complement}$?

Rezolvare

Presupunem prin absurd, $A^{\complement} \in \mathcal{RE}$. $\exists P_A, P_{A^{\complement}} \in \mathcal{P}$ astfel încât:

$$P_A(x) = \begin{cases} 1 & \text{dacă } x \in A \\ \perp & \text{altfel} \end{cases}$$

[1] Mulțimea B descrie problema opririi, care este semi-decidabilă.

$$P_{A^{\complement}}(x) = \begin{cases} 1 & \text{dacă } x \in A^{\complement} \\ \perp & \text{altfel} \end{cases}$$

Construim $Q_A \in \mathcal{P}$ care decide apartenența la A.

```
bool QA(int x) {
  for (Integer t = 0; ; t++) {
    if (run_in_steps(PA, x, t) == true)
      return true;

    if (run_in_steps(PAC, x, t) == true)
      return false;
  }
}
```

$\Rightarrow A \in \mathcal{R}$. Contradicție!

$$A \in \mathcal{RE} \setminus \mathcal{R} \Rightarrow A^{\complement} \notin \mathcal{RE}$$

Exercițiul 5

Fie mulțimea $A \notin \mathcal{RE}$, și $B \subset A$ astfel încât $A \setminus B \in \mathcal{R}$. Ce putem spune despre B?

Rezolvare

Presupunem prin absurd $B \in \mathcal{RE} \Rightarrow \exists P_{AminusB}, P_B \in \mathcal{P}$, astfel încât

$$P_{AminusB}(x) = \begin{cases} 1 & \text{dacă } x \in A \setminus B \\ 0 & \text{altfel} \end{cases}$$

$$P_B(x) = \begin{cases} 1 & \text{dacă } x \in B \\ \perp & \text{altfel} \end{cases}$$

Atunci putem construi $P_A \in \mathcal{P}$ care afirmă aparteneța la mulțimea A.

```
bool PA(int x) {
  if (PA_minus_B(x))
    return true; // x face parte din A
  return Pb(x); // B inclus în A
}
```

Deci $A \in \mathcal{RE}$. Contradicție! $\Rightarrow B \notin \mathcal{RE}$

Exercițiul 6

Fie $A \in \mathcal{RE}$, și mulțimea $B = \{x + 1 | x \in A\}$. Dacă $A \cap B = \emptyset$ și $A \cup B \in \mathcal{R}$ demonstrați că $A, B \in \mathcal{R}$.

Rezolvare

Intuim că $B \in \mathcal{RE}$.

```
bool PB(int x) {
  return PA(x - 1);
}
```

$\Rightarrow B \in \mathcal{RE}$. Rezolvarea mai departe este asemănătoare cu exercițiul 1. Următoarele programe decid apartenența la mulțimile A și B.

```
bool QA(int x) {
  if (!PA_plus_B(x))
    return false;
  for (Integer t = 0; ; t++) {
    if (run_in_steps(PA, x, t) == true)
      return true;

    if (run_in_steps(PB, x, t) == true)
      return false;
  }
}

bool QB(int x) {
  if (!PA_plus_B(x)) // prog. care decide apartenența la reuniune
    return false;
  for (Integer t = 0; ; t++) {
    if (run_in_steps(PA, x, t) == true)
      return false;

    if (run_in_steps(PB, x, t) == true)
      return true;
  }
}
```

Exercițiul 7

Fie $A \subset N$ care nu are nicio pereche de elemente consecutive și $B = \{x+1 | x \in A\}$. Demonstrați că dacă $A \cup B$ recursivă atunci și A și B sunt recursive.

Rezolvare

Observație: $\forall x \in A, x+1 \notin A$, această regulă se respectă și pentru B. (Demonstrația se poate face prin reducere la absurd.)

Acum, trebuie să ne imaginăm cum sunt distribuite elementele din A, respectiv din B, în mulțimea $A \cup B$.

Dacă $x+1 \in A \cup B$, atunci:

$$x+1 \in \begin{cases} B & \text{dacă } x \in A \\ A & \text{altfel} \end{cases}$$

Dacă $x \in A \cup B$, atunci $y_x = max(\{n \in \mathbb{N} | n \notin A \cup B, n < x\})$ (cu alte cuvinte, y_x fiind cel mai mare număr mai mic decât x care nu aparține reuniunii), $y_x + 1 \in A$. (Altfel s-ar contrazice paragraful anterior și $y_x \in A \cup B$, contradicție!)

Următoarul program decide apartenența la mulțimea A.

```
bool PA(x) {
  Boolean prev = null;

  // căutăm primul nr. care nu aparține reuniunii
  int i = x;
  while (!PA_plus_B(i))
    i--;

  // cel mai mare număr mai mic decât x care nu aparține reuniunii este i
  bool res = false;
  for (int j = i + 1; j <= x; j++)
    res = !res;

  return res;
}
```

Asemenea se poate construi $Q_B \in \mathcal{P}$ pentru B.

Exercițiul 8

Mulțimea $A \notin \mathcal{RE}$. Ce putem spune despre complementul lui A, $A^{\complement}$?

Rezolvare

Presupunem prin absurd $A^{\complement} \in \mathcal{R}$. Precum $\mathcal{R}$ este închisă pe operația de complement, rezultă că $A \in \mathcal{R}$, contradicție!

Presupunem prin absurd $A^{\complement} \in \mathcal{RE}$ și o să contrazicem presupția printr-un exemplu.

Caz particular cunoscut

Fie $A = \{P \in \mathcal{P} | \forall x \in \mathbb{N}, P(x) \neq \perp\}$ și $A^{\complement} = \{P \in \mathcal{P} | \exists x \in \mathbb{N}, P(x) = \perp\}$.

Cu alte cuvinte A este mulțimea tuturor programelor care se termină pe orice input, iar $A^{\complement}$ este mulțimea programelor pentru care există cel puțin un input care duce la o execuție fără sfârșit.

Încă o reformulare poate fi: orice program din A, decide o problemă.[2]

Se poate demonstra că $A, A^{\complement} \notin \mathcal{RE}$. (Carl Mummert, f.a.)

Observație

$A = \{(P, x) \in \mathcal{P} \times \mathbb{N} | P(x) = \perp\}$, iar $A^{\complement}$ desemnează problema opririi, $A^{\complement} \in \mathcal{RE} \setminus \mathcal{R}$.

Deci putem spune doar că $A^{\complement} \notin \mathcal{R}$.

Bibliografie

Carl Mummert. f.a. „Show that the language TOT is not recursively enumerable nor its complement." Mathematics Stack Exchange. https://math.stackexchange.com/q/578107.

[2] Mulțimea A este un exemplu cunoscut de Π_2-completitudine.

Seminar 2 - Exerciții rezolvate

Echipa AA

23 octombrie 2022

Cuprins

Notații **1**

Recapitulare teorie **1**
Exemplu, problema opririi (eng: Halting Problem) . . . 2
Reduceri Turing . . . 2
Consecințe . . . 3

Exemplul 1 - VID (din seminar, rescris) **3**

Exercițiul 1 - PO_5 **3**
Pasul 1 . . . 4
Pasul 2 . . . 4

Exercițiul 2 - $DOAR_5$ **4**

Exercițiul 3 - EXP_2 **5**

Exercițiul 4 - $STOP_3$ **6**

Exercițiul 5 - EQP **6**
$VID \leq_T EQP$ . . . 7
$PO \leq_T EQP$ . . . 7

Bibliografie **8**

Notații

În acest document vom folosi următoarele notații:

- $\mathcal{R}$ - mulțimea tuturor mulțimilor recursive
- $\mathcal{RE}$ - mulțimea tuturor mulțimilor recursive-enumerabile
- $\mathcal{P}$ - mulțimea tuturor programelor.
- $\mathcal{I}$ - o mulțime arbitrară de intrări (sau, pe englezește, inputuri)

Vom nota descrierile problemelor, cu litere mari latine.
Predicatul corelat unei probleme îl vom scrie cu litere mici grecești indice numele problemei.
Programul corelate unei probleme îl vom scrie cu majusculă și litere latine indice numele problemei.
Mulțimea corelată unei probleme o vom scrie cu cu majusculă și litere ebraice indice numele problemei.

Recapitulare teorie

O problemă de decizie este o întrebare arbitrară pe o mulțime specifică de intrări la care se poate răspunde binar, "da" sau "nu", "1" sau "0".[1]

O să atașăm unei probleme de decizie, $PROB$, un predicat (o funcție), $\pi_{PROB} : \mathcal{I} \to \{0, 1\}$, astfel:

[1] Definirea formală a unei probleme se poate face în mai multe feluri, vezi Colorni și Tsoukiàs (2013) și Kroening și Strichman (2016), dar noi o să folosim abordarea din Giumale (2004).

$$\pi_{PROB}(i) = \begin{cases} 1 & \text{dacă răspunsul la problema } PROB \text{ pentru } i \text{ e "da"} \\ 0 & \text{dacă răspunsul la problema } PROB \text{ pentru } i \text{ e "nu"} \end{cases}$$

Notăm cu $\aleph_{PROB}$ mulțimea de adevăr a lui π_{PROB}:

$$\aleph_{PROB} = \{i \in \mathcal{I} \mid \pi_{PROB}(i) = 1\}$$

Atenție: Unui predicat s-ar putea să îi corespundă, sau nu, un program care oferă un răspuns binar pentru problema descrisă.

Exemplu, problema opririi (eng: Halting Problem)

> *PO*: Fiind dat un program arbitrat și o intrare oarecare pentru el, se va opri programul pe acea intrare?

Atașăm problemei opririi, PO, predicatul/funcția

$$\pi_{PO}((P, w)) = \begin{cases} 1 & P(w) \text{ se oprește} \\ 0 & \text{altfel} \end{cases}$$

Cu alte cuvinte predicatul/funcția primește la intrare un cuplu, format dintr-un program, $P \in \mathcal{P}$, și o intrare oarecare asociată programului P, intrarea $w \in \mathcal{I}$. Predicatul spune pentru acest cuplu un rezultat binar, care e răspunsul problemei opririi. Deoarece scrierea sub formă de tuplu (P, w) este anevoiasă, vom rescrie predicatul/funcția ca primind mai mulți parametrii când e nevoie.

Atenție: Pentru predicatul/funcția, π_{PO}, nu există un program (o funcție efectiv calculabilă), care să calculeze răspunsul la problema opririi, adică

$$\nexists Q \in \mathcal{P}, Q(P, w) = \begin{cases} 1 & P(w) \text{ se oprește} \\ 0 & \text{altfel} \end{cases}$$

Asociată predicatului π_{PO}, notăm cu $\aleph_{PO}$ mulțimea de adevăr a problemei opririi, PO.

$$\aleph_{PO} = \{(P, w) \in \mathcal{P} \times \mathcal{I} \mid \pi_{PO}(P, w) = 1\}$$

Reduceri Turing

Fie două predicate asociate la două probleme, $PROB_1$ și $PROB_2$:

$$\pi_{PROB_A} : \mathcal{I}_{\mathcal{A}} \to \{0, 1\}$$
$$\pi_{PROB_B} : \mathcal{I}_{\mathcal{B}} \to \{0, 1\}$$

Spunem că $PROB_A$ se reduce la $PROB_B$, dacă $\exists f : \mathcal{I}_A \to \mathcal{I}_B$, o funcție efectiv calculabilă[2] (și totală[3]), astfel încât:

$$\forall i_A \in \mathcal{I}_A, \pi_{PROB_A}(i_A) = 1 \iff \pi_{PROB_B}(\underbrace{f(i_A)}_{i_b}) = 1$$

și se notează așa[4]:

$$PROB_A \leq_T PROB_B$$

Atenție: Se mai practică notația:

$$\aleph_{PROB_A} \leq_T \aleph_{PROB_B}, \text{dacă}$$

$$\forall i_A \in \mathcal{I}_{\mathcal{A}}, i_A \in \aleph_{PROB_A} \iff \underbrace{f(i_A)}_{i_b} \in \aleph_{PROB_B}$$

[2] recursivă, vezi Post et al. (1944)

[3] Adică funcția este definită pentru toate elementele din domeniu

[4] Indicele "T" vine de la Turing. Cu toate acestea noi **NU** facem o reducere precum a definit-o el în teza lui de doctorat (vezi Turing (1939)). În cel mai strict sens noi facem o reducere prin mapare, de la mai multe la unu, notată în cărțile de specialitate $\leq_m$. Aceasta este mai specifică (vezi Soare (2016)), decât formularea generală introdusă de Post et al. (1944) a reducerii generale Turing.

Consecințe

$$\aleph \leq_T \beth$$

1. $\beth \in \mathcal{R} \Rightarrow \aleph \in \mathcal{R}$
2. $\beth \in \mathcal{RE} \Rightarrow \aleph \in \mathcal{RE}$
3. $\aleph \notin \mathcal{R} \Rightarrow \beth \notin \mathcal{R}$
4. $\aleph \notin \mathcal{RE} \Rightarrow \beth \notin \mathcal{RE}$

Exemplul 1 - VID (din seminar, rescris)

> VID: Are un program arbitrar proprietatea de a nu se opri pe nicio intrare?

Intuim[5] că problema VID nu nu este semidecidabilă, adică $\aleph_{VID} \notin \mathcal{RE}$.
Folosim consecința 4, deci vom reduce o problemă nici măcar semidecidabilă, $\overline{PO}$, la VID.

$$\overline{PO} \leq_T VID$$

Fie predicatele asociate acestor probleme:

$$\pi_{\overline{PO}}(P, w) = \begin{cases} 1 & P(w) \textbf{ nu} \text{ se oprește} \\ 0 & \text{altfel} \end{cases}$$

$$\pi_{VID}(Q) = \begin{cases} 1 & Q \text{ nu se oprește pe nicio intrare} \\ 0 & \text{altfel} \end{cases}$$

Atenție: Programele date ca argument acestor predicate, P și Q, nu au nicio legătură între ele. Sunt numele a două programe oarecare din $\mathcal{P}$.

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P}$$
$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{\overline{PO}}(P, w) = 1 \iff \pi_{VID}(\underbrace{f(P, w)}_{Q}) = 1$$

Adică funcția f[6] trebuie să construiască, pentru orice cuplu program-intrare care nu se oprește, un alt program, Q, care nu se oprește pe nicio intrare.

```
1 void Q(GenericInput v) { // își ignoră inputul
2   P(w);
3   return; // și așa în programul se oprea dacă trecea de apelul anterior
4 }
```

Demonstrăm dubla implicație pe rând

1. $\pi_{\overline{PO}}(P, w) = 1 \Rightarrow \pi_{VID}(\underbrace{f(P, w)}_{Q}) = 1$

   Dacă $P(w)$ nu se oprește $\Rightarrow Q$, nu se oprește pe nicio intrare.

2. $\pi_{VID}(\underbrace{f(P, w)}_{Q}) = 1 \Rightarrow \pi_{\overline{PO}}(P, w) = 1$

   Dacă Q nu s-a oprit pe nicio intrare, înseamnă că nu s-a ajuns niciodată la linia 3 din codul de mai sus $\Rightarrow P(w)$ nu se oprește.

Exercițiul 1 - PO_5

[5] S-ar putea ca intuiția să ne și înșele. Ea provine de la faptul că avem de verificat o proprietate pentru o infinitate de intrări și problema neopririi nu este semidecidabilă.

[6] Construirea unui alt program dat fiind un program inițial la intrare este o sarcină pe care compilatoarele o duc la capăt zilinc. Cu toate acestea ca să aratăm că funcția este efectiv calculabilă ar trebuie să folosim niște aspecte pe care le veți studia la materia "Limbaje Formale și Automate".

> PO_5: Se termină un program arbitrar pe intrarea 5?

Intuim că problema PO_5 este semidecidabilă, adică $\aleph_{PO_5} \in \mathcal{RE} \setminus \mathcal{R}$.

Pasul 1

Demonstrăm mai întâi, cu cunoștiințele din seminarul trecut, că $\aleph_{PO_5} \in \mathcal{RE}$. Scriem programul Sim_{PO_5}, astfel încât:

$$Sim_{PO_5}(Q) = \begin{cases} 1 & \text{dacă } Q(5) \text{ se termină} \\ \perp & \text{altfel} \end{cases}$$

```
bool Sim_PO_5(NumericProgram Q) { // NumericProgram - tipul unei funcții cu un input numeric
  Q(5);
  return true;
}
```

Pasul 2

Demonstrăm folosind consecința 3, deci vom reduce o problemă semidecidabilă, $PO \notin \mathcal{R}$, la PO_5.

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P}$$
$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{PO}(P, w) = 1 \iff \pi_{PO_5}(\underbrace{f(P, w)}_{Q}) = 1$$

Adică funcția f trebuie să construiască, pentru orice cuplu program-intrare care se oprește, un alt program, Q, care se oprește pe intrarea 5.

```
void Q(Number x) { // își ignoră inputul
  P(w);
}
```

Demonstrăm dubla implicație pe rând

1. $\pi_{PO}(P, w) = 1 \Rightarrow \pi_{PO_5}(\underbrace{f(P, w)}_{Q}) = 1$

   Dacă $P(w)$ se oprește $\Rightarrow Q$, se oprește pe intrarea 5.

2. $\pi_{PO_5}(\underbrace{f(P, w)}_{Q}) = 1 \Rightarrow \pi_{PO}(P, w) = 1$

   Dacă Q se oprește pe 5, înseamnă că s-a ajuns la linia 3 din codul de mai sus $\Rightarrow P(w)$ se oprește.

Exercițiul 2 - $DOAR_5$

> $DOAR_5$: Se termină un program arbitrar **doar** pe intrarea 5?

Intuim că problema $DOAR_5$ nu este semidecidabilă, adică $\aleph_{DOAR_5} \notin \mathcal{RE}$. Fie predicatul ei:

$$\pi_{DOAR_5}(Q) = \begin{cases} 1 & Q \text{ se oprește doar pe intrarea 5} \\ 0 & \text{altfel} \end{cases}$$

Folosim consecința 4, deci vom reduce o problemă nici măcar semidecidabilă, $\overline{PO}$, la $DOAR_5$.

$$\overline{PO} \leq_T DOAR_5$$

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P}$$

$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{\overline{PO}}(P, w) = 1 \iff \pi_{DOAR_5}(\underbrace{f(P, w)}_{Q}) = 1$$

Adică funcția f trebuie să construiască, pentru orice cuplu program-intrare care nu se oprește, un alt program, Q, care se oprește pe doar pe 5.

```
1  void Q(Number w) {
2    if (w == 5)
3      return;
4    else
5      P(w);
6  }
```

Demonstrăm dubla implicație pe rând

1. $\pi_{\overline{PO}}(P, w) = 1 \Rightarrow \pi_{DOAR_5}(\underbrace{f(P, w)}_{Q}) = 1$

   Dacă $P(w)$ nu se oprește $\Rightarrow Q$ se va opri doar dacă intră pe prima ramură a `if`-ului, deci se va opri doar pe 5.

2. $\pi_{DOAR_5}(\underbrace{f(P, w)}_{Q}) = 1 \Rightarrow \pi_{\overline{PO}}(P, w) = 1$

   Dacă Q s-a oprit doar pe 5, s-a oprit doar prin apelul `return;` de la linia 3 $\Rightarrow P(w)$ nu se oprește.

Exercițiul 3 - EXP_2

> EXP_2: Calculează un program arbitrar 2^x pentru orice intrare $x \in \mathbb{N}$?

Intuim că problema EXP_2 nu este semidecidabilă, adică $\aleph_{EXP_2} \notin \mathcal{RE}$. Fie predicatul ei:

$$\pi_{EXP_2}(Q) = \begin{cases} 1 & \forall x \in \mathbb{N}, Q(x) = 2^x \\ 0 & \text{altfel} \end{cases}$$

Folosim consecința 4, deci vom reduce o problemă nici măcar semidecidabilă, $\overline{PO}$, la EXP_2.

$$\overline{PO} \leq_T EXP_2$$

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P}$$
$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{\overline{PO}}(P, w) = 1 \iff \pi_{EXP_2}(\underbrace{f(P, w)}_{Q}) = 1$$

Adică funcția f trebuie să construiască, pentru orice cuplu program-intrare care nu se oprește, un alt program, Q, astfel încât $\forall x \in \mathbb{N}, Q(x) = 2^x$.

```
1  // rulează funcția P asupra inputului x în steps pași
2  // dacă funcția a returnat în mai puțini pași, returnează valoarea ei
3  // altfel returnează o referință nulă
4  Boolean run_in_steps(BooleanFunction P, Input x, Integer steps);
5
6  Number Q(Number x) {
7    if (run_in_steps(P, w, x) != null)
8      while (1);
9    else
10     return pow(2, x);
11 }
```

Demonstrăm dubla implicație pe rând

1. $\pi_{\overline{PO}}(P, w) = 1 \Rightarrow \pi_{EXP_2}(\underbrace{f(P, w)}_{Q}) = 1$

   Dacă $P(w)$ nu se oprește $\Rightarrow \nexists x \in \mathbb{N}$ astfel încât Q să se oprească în x pași, deci se va ajunge mereu la linia 10.

2. $\pi_{EXP_2}(\underbrace{f(P, w)}_{Q}) = 1 \Rightarrow \pi_{\overline{PO}}(P, w) = 1$

   Dacă se ajunge mereu la linia 10 $\Rightarrow \forall x \in \mathbb{N}, P(w)$ nu se oprește în x pași, deci $P(w)$ nu se oprește.

Exercițiul 4 - $STOP_3$

> $STOP_3$: Se oprește un program arbitrar pe fix 3 intrari numerice?

Intuim că problema $STOP_3$ nu este semidecidabilă, adică $\aleph_{STOP_3} \notin \mathcal{RE}$. Fie predicatul ei:

$$\pi_{STOP_3}(Q) = \begin{cases} 1 & \text{Q se oprește pe fix 3 intrări} \\ 0 & \text{altfel} \end{cases}$$

Folosim consecința 4, deci vom reduce o problemă nici măcar semidecidabilă, $\overline{PO}$, la $STOP_3$.

$$\overline{PO} \leq_T STOP_3$$

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P}$$
$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{\overline{PO}}(P, w) = 1 \iff \pi_{STOP_3}(\underbrace{f(P, w)}_{Q}) = 1$$

Adică funcția f trebuie să construiască, pentru orice cuplu program-intrare care nu se oprește, un alt program, Q, care se oprește pe fix 3 intrări.

```
void Q(Number x) {
  if (0 <= x && x < 3)
    return;
  else
    P(w);
}
```

Demonstrăm dubla implicație pe rând

1. $\pi_{\overline{PO}}(P, w) = 1 \Rightarrow \pi_{STOP_3}(\underbrace{f(P, w)}_{Q}) = 1$

   Dacă $P(w)$ nu se oprește $\Rightarrow Q$, se va opri doar când intră pe prima ramură a `if`-ului.

2. $\pi_{STOP_3}(\underbrace{f(P, w)}_{Q}) = 1 \Rightarrow \pi_{\overline{PO}}(P, w) = 1$

   Dacă Q se oprește doar pe 3 intrări, înseamnă că se termină doar pe prima ramură a `if`-ului $\Rightarrow P(w)$ nu se oprește.

Exercițiul 5 - EQP

> EQP: Se opresc două programe arbitrare pe aceleași intrări? (programele primesc același tip de input)
> Reduceți atât PO, cât și VID la EQP.

Scriem predicatul logic asociat problemei:

$$\pi_{EQP}(Q_1, Q_2) = \begin{cases} 1 & \forall i \in \mathcal{I}, Q_1(i) \neq \perp \iff Q_2(i) \neq \perp \\ 0 & \text{altfel} \end{cases}$$

$VID \leq_T EQP$

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \to \mathcal{P} \times \mathcal{P}$$
$$\forall P \in \mathcal{P}, \quad \pi_{VID}(P) = 1 \iff \pi_{EQP}(\underbrace{f(P)}_{(Q_1,Q_2)}) = 1$$

Adică funcția f trebuie să construiască, pentru orice program care se oprește, un cuplu de două programe, (Q_1, Q_2) care se opresc sau nu în tandem.

```
void Loop(GenericInput v) { // își ignoră inputul
  while (1);
}

void Wrapper(GenericInput v) {
  P(w);
}
```

$$Q_1 = Loop, Q_2 = Wrapper$$

Demonstrăm dubla implicație pe rând

1. $\pi_{VID}(P) = 1 \Rightarrow \pi_{EXP}(\underbrace{f(P)}_{(Q_1,Q_2)}) = 1$

   Dacă P nu se oprește pe nicio intrare $\Rightarrow, Wrapper$ se oprește pe toate intrările pe care se oprește și $Loop$.

2. $\pi_{EXP}(\underbrace{f(P)}_{(Q_1,Q_2)}) = 1 \Rightarrow \pi_{VID}(P) = 1$

   Dacă $Wrapper$ se oprește pe toate intările pe care se oprește și $Loop \Rightarrow P$ nu se oprește pe nicio intrare.

$PO \leq_T EQP$

Pentru a face reducerea trebuie să găsim o funcție de mapare efectiv calculabilă, care satisface constrângerile:

$$f : \mathcal{P} \times \mathcal{I} \to \mathcal{P} \times \mathcal{P}$$
$$\forall (P, w) \in \mathcal{P} \times \mathcal{I}, \quad \pi_{PO}(P, w) = 1 \iff \pi_{EQP}(\underbrace{f(P, w)}_{(Q_1,Q_2)}) = 1$$

Adică funcția f trebuie să construiască, pentru orice cuplu program-intrare care se oprește, un cuplu de două programe, (Q_1, Q_2) care se opresc sau nu în tandem.

```
void All(GenericInput v) { // își ignoră inputul
  return; // se oprește mereu
}

void Wrapper(GenericInput v) {
  P(w);
}
```

$$Q_1 = All, Q_2 = Wrapper$$

Demonstrăm dubla implicație pe rând

1. $\pi_{PO}(P, w) = 1 \Rightarrow \pi_{EXP}(\underbrace{f(P, w)}_{(Q_1,Q_2)}) = 1$

   Dacă $P(w)$ se oprește $\Rightarrow Wrapper$ se oprește pe aceleași intrări ca și All.

2. $\pi_{EXP}(\underbrace{f(P, w)}_{(Q_1,Q_2)}) = 1 \Rightarrow \pi_{PO}(P, w) = 1$

   Dacă $Wrapper$ se oprește pe toate intrările pe care se oprește și $All \Rightarrow P(w)$ se oprește.

Bibliografie

Colorni, Alberto, și Alexis Tsoukiàs. 2013. „What is a decision problem? Preliminary statements". În *International Conference on Algorithmic Decision Theory*, 139–53. Springer.

Giumale, Cristian A. 2004. „Introducere în ANALIZA ALGORITMILOR". *Editura Polirom, Bucuresti*.

Kroening, Daniel, și Ofer Strichman. 2016. *Decision procedures*. Springer.

Post, Emil L et al. 1944. *Recursively enumerable sets of positive integers and their decision problems*. World Scientific. https://projecteuclid.org/journals/bulletin-of-the-american-mathematical-society/volume-50/issue-5/Recursively-enumerable-sets-of-positive-integers-and-their-decision-problems/bams/1183505800.pdf.

Soare, Robert I. 2016. *Turing computability: Theory and applications*. Springer.

Turing, Alan Mathison. 1939. „Systems of logic based on ordinals". *Proceedings of the London Mathematical Society, Series 2* 45: 161–228. https://www.dcc.fc.up.pt/~acm/turing-phd.pdf.

1. $O(g(n)) = \{f : N \to R_+ \,|\, \exists ct. c \in R_+, c > 0, \text{și } n_0 \in Na.i.pt \, \forall n \geq n_0, 0 \leq f(n) \leq cg(n)\}$

   - $f(n) \in O(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c * g(n)$ este o **limită superioară** pentru $f(n)$; algoritmul se va purta mereu mai bine decât această limită;

2. $\Omega(g(n)) = \{f : N \to R_+ \,|\, \exists ct. c \in R_+, c > 0, \text{și } n_0 \in Na.i.pt \, \forall n \geq n_0, 0 \leq cg(n) \leq f(n)\}$

   - $f(n) \in \Omega(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c * g(n)$ este o **limită inferioară** pentru $f(n)$; algoritmul se va purta mereu mai rău decât această limită;

3. $\Theta(g(n)) = \{f : N \to R_+ \,|\, \exists ct. c_1, c_2 \in R_+, c_1 > 0, c_2 > 0 \text{și } n_0 \in Na.i.pt \, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

   - $f(n) \in \Theta(g(n))$ înseamnă că, pentru valori mari ale dimensiunii intrării, $c_1 g(n)$ este o **limită inferioară** pentru $f(n)$, iar $c_2 g(n)$ o **limită superioară**; algoritmul tinde să se comporte "cam ca" $g(n)$.
   - **Pentru analize de complexitate cât mai precise preferăm notația** Θ.

4. $o(g(n)) = \{f : N \to R_+ \,|\, \forall c \in R_+, c > 0, \exists n(c) \in Na.i.pt \, \forall n \geq n(c), 0 \leq f(n) < cg(n)\}$

   - $f(n) \in o(g(n))$ înseamnă că $g(n)$ crește **strict** mai repede decât $f(n)$

5. $\omega(g(n)) = \{f : N \to R_+ \,|\, \forall c \in R_+, c > 0, \exists n(c) \in Na.i.pt \, \forall n \geq n(c), 0 \leq cg(n) < f(n)\}$

   - $f(n) \in \omega(g(n))$ înseamnă că $f(n)$ crește **strict** mai repede decât $g(n)$

Redefinirea notațiilor asimptotice folosind limite de funcții:

- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 \leq L < \infty \Rightarrow f(n) \in O(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 < L < \infty \Rightarrow f(n) \in \Theta(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = L \; cu \; 0 < L \leq \infty \Rightarrow f(n) \in \Omega(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = 0 \Rightarrow f(n) \in o(g(n))$
- $\lim_{n\to\infty}(\frac{f(n)}{g(n)}) = \infty \Rightarrow f(n) \in \omega(g(n))$

2. Demonstrați folosind notațiile asimptotice de complexitate sau cele cu limite:

(a) $5 * \log n + 6 \in O(n)$
(b) $n^2 \in \Theta(n^2 + n)$
(c) $\log_a n \in \Theta(\log_2 n)$
(d) $\sqrt{n} \in \Omega(\log n)$
(e) $O(n^2 + n) = O(n^2)$

a) $\lim_{n\to\infty} \frac{5\log n + 6}{n} \overset{\infty/\infty}{=} \frac{5 \cdot \frac{1}{n}}{1} = 0 \in [0,\infty) \Rightarrow o(n)$

b) $\lim_{n\to\infty} \frac{n^2}{n^2+n} = 1 - \lim_{n\to\infty} \frac{n}{n^2+n} \overset{\infty/\infty}{=} 1 - \lim_{n\to\infty} \frac{1}{2n+1} = 1 \in (0,\infty)$

c) $\lim_{n\to\infty} \frac{\log_a n}{\log_2 n} = \lim_{n\to\infty} \frac{\frac{\log_2 n}{\log_2 a}}{\log_2 n} = \frac{1}{\log_2 a} \in (0,\infty)$ $(\text{pt } a > 1)$

d) $\lim_{n\to\infty} \frac{\sqrt{n}}{\log n} \overset{\infty/\infty}{=} \lim_{n\to\infty} \frac{\frac{1}{2} n^{-1/2}}{n^{-1}} = \frac{1}{2} \lim_{n\to\infty} \sqrt{n} = \infty$ ✓

e) fie $f \in o(n^2+n) \Rightarrow \exists c > 0,\ 0 \le f(n) \le c(n^2+n)$
dacă $f \in O(n^2) \Rightarrow \exists c',\ 0 \le f(n) \le c' n^2$
pt $c' = 2c$, avem $c(n^2+n) \le 2c n^2 \Leftrightarrow n \le n^2$ ✓
$\Rightarrow O(n^2+n) \subseteq o(n^2)$

3. Identificați răspunsul pentru fiecare subpunct:

(a) $\Omega(g(n)) \cap \Theta(g(n)) = ?$ $0 < L \le \infty$ și $0 < L < \infty \Rightarrow 0 < L < \infty \Rightarrow \Theta(g(n))$
(b) $O(g(n)) \cup \Theta(g(n)) = ?$ $0 \le L < \infty$ sau $0 < L < \infty \Rightarrow 0 \le L < \infty \Rightarrow O(g(n))$
(c) $\omega(g(n)) + \Theta(g(n)) = ?$ $\lim \frac{f}{g} = \lim \frac{f+f_1}{g} = \infty + L = \infty \Rightarrow \omega(g(n))$
(d) $2^{2n} \in \Theta(2^n) = ?$ $\lim \frac{2^{2n}}{2^n} = \lim 2^n = \infty \Rightarrow$ fals

4. Rezolvați:

(a) Știind că f(n) ∈ $o(n^2)$ și h(n) ∈ $\Theta(n^3)$, determinați complexitatea f(n) * h(n).
(b) Știind că f(n) ∈ $\Omega(n)$ și h(n) ∈ $O(n^2)$, determinați complexitatea h(n) / f(n).
(c) Știind că f(n) ∈ $O(n^5)$ și g(n) ∈ $\Theta(\sqrt{n})$, este adevărată afirmația: f(g(n)) ∈ $\omega(n^2)$?
(d) Știind că f(n) ∈ $O(g(n))$, care este limita asimptotică pentru: o(f(n)) + o(g(n))?
(e) Știind că f(n) ∈ $O(\log(n))$, este adevărată afirmația: $2^{f(n)}$ ∈ O(n)?

a) $0 \le f(n) < c n^2$
$c_1 n^3 \le f(n) \le c_2 n^3 \quad | \cdot f(n)$
$c_1 n^3 f(n) \le f(n) h(n) \le c_2 n^3 f(n)$
$0 \le \downarrow < c c_1 n^5 \qquad 0 \le \downarrow < c c_2 n^5$
$\Rightarrow 0 \le f(n) h(n) < c c_2 n^5 \Rightarrow f(n) h(n) \in o(n^5)$

b) $c_1 n \le f(n)$; $0 \le h(n) \le c_2 n^2 \quad | / f(n)$
$0 \le h(n)/f(n) \le c_2 n^2 / f(n) \le \frac{c_2}{c_1} n$
$\Rightarrow h(n)/f(n) \in O(n)$

c) $0 \le f(n) \le c n^5 \quad (1)$
$c_1 \sqrt{n} \le g(n) \le c_2 \sqrt{n} \quad (2)$

$(1) \circ g \Rightarrow 0 \le f(g(n)) \le c\, g^5(n)$

$c(c_1\sqrt{n})^5 \le \downarrow \le c_1(c_2\sqrt{n})^5$

$\Rightarrow 0 \le f(g(n)) \le c_1 c_2^5\, n^{5/2}$

deci $f(g(n)) \in \omega(n^2) \Rightarrow c' n^2 < f(g(n)) \rightarrow$ fals

d) $f(n) \in O(g(n))$; $o(f(n)) + o(g(n)) \subseteq ?$

fie $f' \in o(f(n))$, $g' \in o(g(n))$

$0 \le f'(n) < c_1 f(n)$, $0 \le g'(n) < c_2 g(n)$

$\hookrightarrow \Rightarrow 0 \le f(n) \le c_3 g(n) \Rightarrow 0 \le f'(n) < c_1 c_3 g(n)$

$\Rightarrow 0 \le f'(n) + g'(n) < (c_1 c_3 + c_2) g(n)$

$\Rightarrow o(g(n))$

e) $f(n) \in O(\log n)$; e adev $2^{f(n)} \in O(n)$?

$\Leftrightarrow 0 \le f(n) \le c \log n$ | $2^{\cdot}$

$2^0 \le 2^{f(n)} \le 2^{c \log n}$

$1 \le 2^{f(n)} \le n^c$

$\nexists c', n^c \le c' n \Rightarrow$ fals

5. Calculați complexitatea următoarelor bucăți de cod:

(a)
```
for (int i=1; i<c; i++) do
    //operații cu complexitate O(1)
end for
```

$c \cdot O(1) = O(1)$

(b)
```
for (int i=1; i<=n; i+ = c) do
    //operații cu complexitate O(1)
end for
```

$\frac{n}{c} \cdot O(1) = O(n)$

(c)
```
for (int i=n; i>0; i− = c) do
    for (int j=i+1; j<=n; j+ = c) do
        //operații cu complexitate O(1)
    end for
end for
```

$\frac{n-i}{c}$ →

$\sum_0^{n/c} \frac{n}{c} - k = \left(\frac{n}{c}\right)^2 - \frac{n(n+1)}{2c^2} = O(n^2)$

(d)
```
for (int i=n; i>0; i−−) do
    for (int j=1; j<n; j∗ =10) do
        for (int k=0; k<j; k++) do
            ... //număr constant c1 de operații
        end for
    end for
end for
```

$O(1)$ →

$n\left(\sum_0^{\log_{10} n} 10^k\right) = n \cdot \frac{10n-1}{10-1} = O(n^2)$

(e)
```
for (int i=1; i<n; i∗ =2) do
    for (int j=n; j>0; j/ =2) do
        for (int k=j; k<n; k++) do
            ... //număr constant c2 de operații
        end for
    end for
end for
```

$O(1)$ →

$\log_2 n \cdot \left(\sum_0^{\log_2 n} n - \frac{n}{2^k}\right) =$

$= \log_2 n \left(n \log_2 n - \frac{n(2n-1)}{2n-1}\right)$

$= \log_2 n (n \log_2 n - n)$

$= O(n \log^2 n)$

(f)
```
for (int i=1; i<=n; i* =2) do
    for (int j=0; j<i; j++) do
        for (int k=0; k<n; k+ =2) do
            ... //număr constant c3 de operații
        end for
        for (int k=1; k<n; k* =2) do
            ... //număr constant c4 de operații
        end for
```

$\sum_{k=0}^{\log_2 n} 2^k = \frac{2n-1}{2-1} = 2n-1$

$a(1) \rightarrow$ ... $\rightarrow (1)$

$a(1) \rightarrow$ ... $\rightarrow (2)$

$(1): \frac{n}{2}(2n-1) = n^2 - \frac{n}{2}$

$(2): \log_2 n (2n-1)$

$(1)+(2) \leq n^2 + \ldots \in \Theta(n^2)$

Seminar 4 - Exerciții rezolvate

Echipa AA

06 noiembrie 2022

Cuprins

Exercițiul 1 **1**

Exercițiul 2 **1**

Exercițiul 3 **2**

Exercițiul 4 **2**

Exercițiul 5 **3**

Exercițiul 7 **4**

Exercițiul 12 **4**

Exercițiul 13 **5**

Exercițiul 14 **5**

Bibliografie **5**

Exercițiul 1

> $T(n) = 2 \cdot T(n-1) + 1$

$$
\begin{aligned}
T(n) &= 2 \cdot T(n-1) && +1 \\
2 \cdot T(n-1) &= 2^2 \cdot T(n-2) && +2 \\
2^2 \cdot T(n-2) &= 2^3 \cdot T(n-3) && +2^2 \\
&\dots \\
2^k \cdot T(n-k) &= 2^{k+1} \cdot T(n-(k+1)) && +2^k \\
&\dots \\
2^{n-1} \cdot T(n-(n-1)) &= 2^n \cdot \underbrace{T(0)}_{\Theta(1)} && +2^{n-1}
\end{aligned}
$$

$$\sum_{k=0}^{n-1} 2^k = \frac{2^n - 1}{2 - 1} = 2^n - 1$$

$$\Rightarrow T(n) \in \Theta(2^n + 2^n - 1) = \Theta(2^n)$$

Exercițiul 2

> $T(n) = T(n-2) + \Theta(n)$

Notăm

- $h = \lfloor \frac{n}{2} \rfloor$.

- $f(n) = c \cdot n \in \Theta(n)$, unde c este o constantă, **fixată**.[1]

$$\begin{aligned}
T(n) &= T(n-2) && +c \cdot n \\
T(n-2) &= T(n-4) && +c(n-2) \\
T(n-4) &= T(n-6) && +c(n-4) \\
&\dots \\
T(n-2k) &= T(n-2(k+1)) && +c(n-2k) \\
&\dots \\
T(n-2(h-1)) &= \underbrace{T(n-2h)}_{\Theta(1)} && +c(n-2(h-1))
\end{aligned}$$

$$\sum_{k=0}^{h-1} c(n-2k) = h \cdot c \cdot n - 2c \cdot \sum_{k=0}^{h-1} k = h \cdot c \cdot n - c \cdot (h-1) \cdot h =$$
$$= c \cdot h \cdot (n-h+1) = c \cdot \left\lfloor \frac{n}{2} \right\rfloor \cdot (n - \left\lfloor \frac{n}{2} \right\rfloor + 1)$$
$$\Rightarrow T(n) \in \Theta(n^2)$$

Exercițiul 3

$T(n) = T(n-k) + \Theta(n),\ k$ const

Notăm:

- $h = \lfloor \frac{n}{k} \rfloor$.
- $f(n) = c \cdot n \in \Theta(n)$, unde c este o constantă, **fixată**.

$$\begin{aligned}
T(n) &= T(n-k) && +c \cdot n \\
T(n-k) &= T(n-2k) && +c(n-k) \\
T(n-2k) &= T(n-3k) && +c(n-2k) \\
&\dots \\
T(n-l \cdot k) &= T(n-(l+1) \cdot k) && +c(n-l \cdot k) \\
&\dots \\
T(n-(h-1) \cdot k) &= \underbrace{T(n-h \cdot k)}_{\Theta(1)} && +c(n-(h-1) \cdot k)
\end{aligned}$$

$$\sum_{l=0}^{h-1} c(n-l \cdot k) = h \cdot c \cdot n - c \cdot k \cdot \sum_{l=0}^{h-1} l = h \cdot c \cdot n - c \cdot k \cdot \frac{(h-1) \cdot h}{2}$$
$$= c \cdot h \cdot (n - \frac{k}{2} \cdot h + \frac{k}{2}) = c \cdot \left\lfloor \frac{n}{k} \right\rfloor \cdot (n - \frac{k}{2} \cdot \left\lfloor \frac{n}{k} \right\rfloor + \frac{k}{2})$$
$$\Rightarrow T(n) \in \Theta(n^2)$$

Exercițiul 4

$T(n) = T(n-a) + T(a) + \Theta(n), a \geq 1$
$T(a) \in \Theta(1)$

$$\begin{aligned}
T(n) &= T(n-a) + T(a) + \Theta(n) = T(n-a) + \Theta(1) + \Theta(n) \\
&= T(n-a) + \Theta(n)
\end{aligned}$$

Ca la exercițiul 3, pentru $k = a$.

[1] Această simplificare este acceptată, vezi Cormen et al. (2009), cap. 4.4

Exercițiul 5

$T(n) = 2T(\frac{n}{3}) + n \cdot log(n)$
$T(1), T(2) \in \Theta(1)$

Identificăm:

- $h = \lfloor log_3(n) \rfloor$

$$
\begin{aligned}
T(n) &= 2T(\frac{n}{3}) && +n \cdot log(n) \\
2T(\frac{n}{3}) &= 2^2T(\frac{n}{3^2}) && +\frac{2}{3}n \cdot log(\frac{n}{3}) \\
2^2T(\frac{n}{3^2}) &= 2^3T(\frac{n}{3^3}) && +(\frac{2}{3})^2n \cdot log(\frac{n}{3^2}) \\
&\dots \\
2^kT(\frac{n}{3^k}) &= 2^{k+1}T(\frac{n}{3^{k+1}}) && +(\frac{2}{3})^kn \cdot log(\frac{n}{3^k}) \\
&\dots \\
2^{h-1}T(\frac{n}{3^{h-1}}) &= 2^h \underbrace{T(\frac{n}{3^h})}_{\Theta(1)} && +(\frac{2}{3})^{h-1}n \cdot log(\frac{n}{3^{h-1}})
\end{aligned}
$$

$$
\begin{aligned}
T(n) &= 2^h + \sum_{k=0}^{h-1}(\frac{2}{3})^k n \cdot log(\frac{n}{3^k}) = 2^h + n\sum_{k=0}^{h-1}(\frac{2}{3})^k \cdot (log(n) - log(3^k)) \\
&= 2^h + n \cdot log(n)\sum_{k=0}^{h-1}(\frac{2}{3})^k - log(3)\sum_{k=0}^{h-1}(\frac{2}{3})^k \cdot k
\end{aligned}
$$

Folosim:

$$\sum_{k=0}^{h-1}(\frac{2}{3})^k = \frac{(\frac{2}{3})^h - 1}{\frac{2}{3} - 1} = 2 - 3(\frac{2}{3})^h$$

Fie:

$$S(n) = \sum_{k=1}^{n}(\frac{2}{3})^k \cdot k$$

Rescriem această sumă în două feluri:

$$
\begin{aligned}
S(n) &= \sum_{k=1}^{n}(\frac{2}{3})^k \cdot (1 + (k-1)) \\
&= \sum_{k=1}^{n}(\frac{2}{3})^k + \frac{2}{3}\sum_{k=1}^{n-1}(\frac{2}{3})^k \cdot (k-1) \\
&= \sum_{k=1}^{n}(\frac{2}{3})^k + \frac{2}{3}\sum_{k=1}^{n-1}(\frac{2}{3})^k \cdot k \\
&= \frac{(\frac{2}{3})^{n+1} - \frac{2}{3}}{\frac{2}{3} - 1} + \frac{2}{3}S(n-1) \\
&= 2 - 3(\frac{2}{3})^{n+1} + \frac{2}{3}S(n-1)
\end{aligned}
$$

Și o mai rescriem astfel:

$$
\begin{aligned}
S(n) &= \sum_{k=1}^{n-1}(\frac{2}{3})^k k + n(\frac{2}{3})^n \\
&= S(n-1) + n(\frac{2}{3})^n
\end{aligned}
$$

Apoi egalăm cele două forme:

$$S(n-1) + n(\frac{2}{3})^n = 2 - 3(\frac{2}{3})^{n+1} + \frac{2}{3}S(n-1)$$
$$\frac{1}{3}S(n-1) = 2 - 3(\frac{2}{3})^{n+1} - n(\frac{2}{3})^n$$
$$S(n-1) = 3(2 - 3(\frac{2}{3})^{n+1} - n(\frac{2}{3})^n)$$
$$S(n-1) = 6 - 3(\frac{2}{3})^n(2+n)$$

$$T(n) = \underbrace{2^h}_{\Theta(n)} + \underbrace{n \cdot log(n) \cdot (2 - 3(\frac{2}{3})^h)}_{\Theta(n \cdot log(n))} - log(3) \cdot (6 - \underbrace{3(\frac{2}{3})^h(h+2)}_{\Theta(n)})$$

$$\Rightarrow T(n) \in \Theta(n \cdot \log n)$$

Exercițiul 7

$T(n) = T(\sqrt{n}) + \log n$
$T(1) \in \Theta(1)$

Facem o schimbare de variabilă:

$$n = 2^m$$
$$m = \log_2 n$$

$$T(2^m) = T(\sqrt{2}^m) + m$$
$$= T(2^{\frac{m}{2}}) + m$$

Notăm:

$$S(m) = T(2^m)$$
$$= S(\frac{m}{2}) + m$$

Știm din recurența cunoscută care modelează algoritmul de sortare prin interclasare că:

$$S(m) \in \Theta(m \log m)$$

Deci:

$$T(n) \in \Theta(\log n \cdot \log\log n)$$

Exercițiul 12

$T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + n^2$
$T(1), T(2), T(3) \in \Theta(1)$

Observăm că funcția este crescătoare, fapt ce se poate demonstra prin inducție.
Deci vom majora și minora termenul general cu niște recurențe mai ușor de calculat.

$$\underbrace{T(\frac{n}{4}) + n^2}_{\Theta(n^2)} \leq T(n) \leq \underbrace{2T(\frac{n}{2}) + n^2}_{\Theta(n^2)}$$

$$\Rightarrow T(n) \in \Theta(n^2)$$

Exercițiul 13

$T(n) = T(\frac{n}{2} + \log_2 n) + n$
$T(1) \in \Theta(1)$

Folosim inegalitatea:

$$\log_2 n \leq \frac{n}{4}, \forall n \geq 16$$

$$\underbrace{T(\frac{n}{2}) + n}_{\Theta(n)} \leq T(n) \leq \underbrace{T(\frac{3n}{4}) + n}_{\Theta(n)}$$

Exercițiul 14

$T(n) = 4T(\frac{n}{2}) + T(\frac{n}{4}) + \frac{n^2}{\log_2 n}$
$T(1), T(2), T(3) \in \Theta(1)$

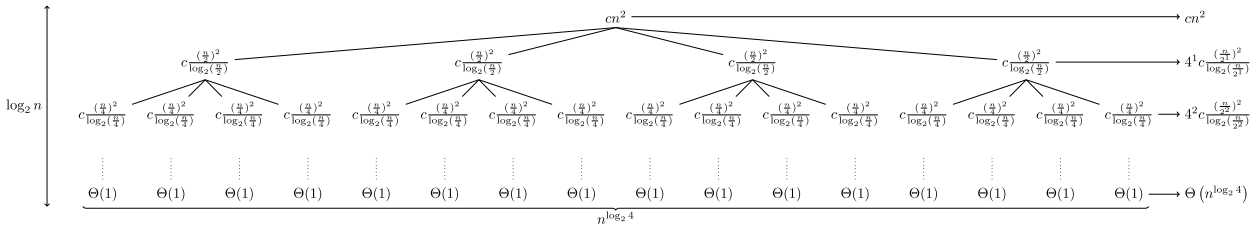


Figura 1: Arborele de recurență pentru $T(n)$

$$T(n) = \Theta(n^{\log_2 4}) + c \sum_{k=0}^{\log_2(n)-1} 4^k \frac{(\frac{n}{2^k})^2}{\log_2(\frac{n}{2^k})}$$

$$T(n) = \Theta(n^2) + c \sum_{k=0}^{\log_2(n)-1} 4^k \frac{\frac{n^2}{4^k}}{\log_2 n - \log_2 2^k}$$

$$= \Theta(n^2) + c \cdot n^2 \sum_{k=0}^{\log_2(n)-1} \frac{1}{\log_2(n) - k}$$

$$= \Theta(n^2) + c \cdot n^2 \underbrace{(1 + \frac{1}{2} + \ldots + \frac{1}{\log_2 n})}_{\Theta(\log\log n)}$$

Am folosit următorul fapt cunoscut:

$$\sum_{i=1}^{m} \frac{1}{i} = \Theta(\log m)$$

$$\Rightarrow T(n) = \Theta(n^2 \log\log n)$$

Bibliografie

Cormen, Thomas H, Charles E Leiserson, Ronald L Rivest, și Clifford Stein. 2009. „Introduction to Algorithms. third". *New York*.

$T(n) = aT(n/b) + f(n)$, unde $a \geq 1, b \geq 1$

1. Dacă $f(n) \in O(n^{\log_b a - \epsilon})$, pentru o constantă $\epsilon > 0$, atunci $T(n) \in \Theta(n^{\log_b a})$.

2. Dacă $f(n) \in \Theta(n^{\log_b a} \log_2^k n)$, cu $k \geq 0$, atunci $T(n) \in \Theta(n^{\log_b a} \log_2^{k+1} n)$.

3. Dacă $f(n) \in \Omega(n^{\log_b a + \epsilon})$, pentru o constantă $\epsilon > 0$ și $af(n/b) \leq cf(n)$ pentru o constantă $c < 1$ și pentru un n suficient de mare, atunci $T(n) \in \Theta(f(n))$.

- $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$
- $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$
- $\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4} \approx \frac{n^4}{4}$
- $\sum_{k=1}^{n} k^\alpha \approx \frac{n^{\alpha+1}}{\alpha+1}$
- $x_{n+1} = qx_n \Rightarrow \sum_{k=1}^{n} x_k = x_0 \frac{q^{n+1}-1}{q-1}$

$\Theta(n \log_2 n)$

1. Arătați că $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n + n \log_2 n)$.

$a = b = 2$, $f \in \Theta(n) \Leftrightarrow f(n) \in \Theta(n^{\log_2 2} \log_2^0 n)$ $\overset{MN2}{\Rightarrow}$ $T(n) \in \Theta(n \log_2 n)$ ✓

(1) $\log_2 a = 1$

2. Rezolvați următoarele recurențe:

(a) $T(n) = 3T(2n/3) + n^3 \log_2 n$

(b) $T(n) = T(n/4) + 1$

(c) $T(n) = 4T(n/2) + n\sqrt{n}$

(d) $T(n) = 16T(n/4) + n!$

(e) $T(n) = 8T(n/3) + 2^n$

(f) $T(n) = T(n/4) + lgn$

(g) $T(n) = 4T(n/2) + \frac{n}{lgn}$

(h) $T(n) = 3T(n/3) + nlgn$

(i) $T(n) = T(n/2 - \log_2 n) + 1$

a) $\log_2 a = \log_{\frac{3}{2}} 3 < 3$

$f(n) \in \Omega(n^{\log_2 a + \epsilon})$

$3\left(\frac{2n}{3}\right)^3 \log_2 \frac{2n}{3} \leq c n^3 \log_2 n$

$3\left(\frac{2}{3}\right)^3 (\log_2 n + \log_2 \frac{2}{3}) \leq c \log_2 n$

$\frac{8}{9} \log_2 \frac{2}{3} \leq \log_2 n \left(c - \frac{8}{9}\right)$

adev $\forall c \in \left(\frac{8}{9}, 1\right)$

$\overset{MN3}{\Rightarrow} T(n) \in \Theta(n^3 \log_2 n)$

b) $\log_2 a = \log_4 1 = 0$

$f(n) = 1 \in \Theta(n^{\log_2 a} \log_2^0 n) \overset{MN2}{\Rightarrow} T(n) \in \Theta(n \log_2 n)$

c) $\log_2 a = \log_2 4 = 2 > 1,5$

$f(n) \in \Omega(n^{\log_2 a - \epsilon}) \overset{MN1}{\Rightarrow} T(n) \in \Theta(n^2)$

d) $\log_2 a = \log_4 16 = 2$

$f(n) = n! \in \Omega(n^2)$

$16\left(\frac{n}{4}\right)! \leq c\, n!$ adev $\forall c \in (0,1)$

$\overset{MN3}{\Rightarrow} T(n) \in \Theta(n!)$

e) $\log_2 a = \log_3 8$

$f(n) = 2^n \in \Omega(n^{\log_3 8})$

$8 \cdot 2^{n/3} \leq c\, 2^n \iff 2^3 \leq c \cdot 2^{2n/3}$ aber $\forall c \in (0,1)$

$\overset{n3}{\Rightarrow} T(n) \notin \Theta(2^n)$

(g) $T(n) = 4T(n/2) + \frac{n}{\lg n}$

(h) $T(n) = 3T(n/3) + n \lg n$

(i) $T(n) = T(n/2 - \log_2 n) + 1$

f) $T(n) = T(n/4) + \log n$

$\log_4 a = 0$

$f(n) = \log n \in \Theta(n^0 \log_2^1 n) \overset{n2}{\Rightarrow} T(n) \in \Theta(\log_2^2 n)$

g) $\log_4 a = \log_4 4 = 2$

$f(n) \in O(n^{2-\varepsilon}) \overset{n3}{\Rightarrow} T(n) \in \Theta(n^2)$

h) $\log_2 a = 1$

$f(n) \in \Theta(n \log_2 n) \overset{n2}{\Rightarrow} T(n) \in \Theta(n \log_2^2 n)$

i) zu $T(n) \in \Theta(\log n)$

$c_1 \log n \leq T(n) \leq c_2 \log n$

für $n_0 = 2$, $T(2) = T(0) + 1 = 1$

$\Rightarrow c_1 \leq 1 \leq c_2$

zu $c_1 \log(n/2 - \log_2 n) \leq T(n/2 - \log_2 n) \leq c_2 \log \ldots n$

$\hookrightarrow ? \; c_1 \log n \leq T(n) \leq c_2 \log n$

$+1 \iff c_1 \log(n/2 - \log_2 n) + 1 \leq T(n) \leq c_2 \log \ldots n + 1$

$c_1 \log n \leq c_1 \log(n/2 - \log n) + 1$

$c_1 \log \frac{n}{n/2 - \log n} \leq 1$

$\lim_{n \to \infty} \frac{n}{n/2 - \log n} = 2 - \lim_{n \to \infty} \frac{2 \log n}{n/2 - \log n} \overset{\frac{\infty}{\infty}}{=} 2 - 2 \lim \frac{\frac{1}{n}}{\frac{1}{2} - \frac{1}{n}} = 2$

$\Rightarrow c_1 \log 2 \leq 1 \Rightarrow c_1 \leq \log^{-1} 2 < 1$

analog, $c_2 \geq 1 > \log^{-1} 2$ ✓

3. Demonstrați că $T(n) = 5T(n/4) + n \in \Theta(5n^{\log_4 5} - 4n)$

$\log_b a < \log_4 5 > 1$

$f(n) = n \in O\left(n^{\log_b a - \varepsilon}\right) \Rightarrow T(n) \in \Theta\left(n^{\log_4 5}\right)$

$\lim_{n \to \infty} \frac{n^{\log_4 5}}{5n^{\log_4 5} - 4n} \overset{\infty}{=} \lim \frac{\log_4 5 \, n^{\log_4 7{,}25}}{5 \log_4 5 \, n^{\log_4 7{,}25} - 4} = \lim \frac{1}{5 - \frac{4}{\log_4 5 \, n^{\log_4 7{,}25}}} = \frac{1}{5}$ ($\frac{4}{\log_4 5 \, n^{\log_4 7{,}25}} = 0$)

$\Rightarrow n^{\log_4 5} \in \Theta(5n^{\log_4 5} - 4n) \Rightarrow T(n) \in$ ...

4. Studiați posibilitatea aplicării teoremei Master in următoarele cazuri:

(a) $T(n) = 2T(n/2) + n \log_2 n$

(b) $T(n) = 3T(n/3) + \frac{n}{\log_2 n}$

a) $\log_b a = 1$

$f(n) = n \log_2 n \in \Theta(n^1 \log_2^1 n) \Rightarrow T(n) \in \Theta(n \log_2^2 n)$

b) $\log_b a = 1$

$f(n) = \frac{n}{\log_2 n}$

caz I: $f(n) \in O(n^{1-\varepsilon})$

$\lim_{n \to \infty} \frac{n/\log_2 n}{n^{1-\varepsilon}} = \lim \frac{n^{\varepsilon}}{\log_2 n} = \infty \Rightarrow$ nu

caz II: $f(n) \in \Theta(n \log_2^{-1} n) : k < 0 \Rightarrow$ nu

caz III: $f(n) \in \Omega(n^{1+\varepsilon})$

$\lim_{n \to \infty} \frac{n/\log_2 n}{n^{1+\varepsilon}} = \lim \frac{1}{n^{\varepsilon} \log_2 n} = 0 \Rightarrow$ nu

$\Rightarrow$ nu se poate :)

5. Rezolvați următoarele recurențe:

(a) $T(n) = 2T(n/2) + 1$

(b) $T(n) = 3T(n/3) + n/2$

(c) $T(n) = 3T(n/3) + n\log_2^2 n$

(d) $T(n) = 3T(n/4) + n\log_2 n$

(e) $T(n) = 5T(n/4) + n$

a) $\log_b a = 1 > 0$

$f(n) = 1 \in O(n^{\log_b a - \varepsilon}) \overset{n1}{\Rightarrow} T(n) \in \Theta(n)$

b) $\log_b a = 1 = 1$

$f(n) = n/2 \in \Theta(n \log_2^a n) \overset{n2}{\Rightarrow} T(n) \in \Theta(n \log_2 n)$

c) $\log_b a = 1 = 1$

$f(n) = n \log_2^2 n \in \Theta(n^1 \log_2^2 n) \overset{n2}{\Rightarrow} T(n) \in \Theta(n \log_2^3 n)$

d) $\log_b a = \log_4 3 < 1$

$f(n) = n \log_2 n \in \Omega(n^{\log_b a + \varepsilon})$

$3\frac{n}{4}\log_2\frac{n}{4} \leq c\, n \log_2 n \Leftrightarrow \frac{3}{4}\log_2 4 \geq \log_2 n\left(\frac{3}{4} - c\right)$

adevărat $c \in [3/4, 1)$

$\overset{n3}{\Rightarrow} T(n) \in \Theta(n \log_2 n)$

e) $\log_b a = \log_4 5 > 1$

$f(n) = n \in O(n^{\log_b a - \varepsilon}) \overset{n1}{\Rightarrow} T(n) \in \Theta(n^{\log_4 5})$

Definiție 2 (Cost amortizat - metoda agregată). În cel mai rău caz, costul mediu sau **costul amortizat** pentru fiecare operație este $\frac{T(n)}{n}$.

→ agregat : $\frac{\text{total}}{n}$

Pentru orice secvență de n operații obținem:

$$\sum_{i=1}^{n} \hat{c}_i \geqslant \sum_{i=1}^{n} c_i$$

- n = numărul de operații din secvența dată
- c_i = **costul real** al operației cu numărul i din secvență
- $\hat{c}_i$ = **costul amortizat** al operației cu numărul i din secvență

credite : alegem $\hat{c}_i$ din burtă a.î. să avem inegalitatea

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Însumând cele n egalități obținem relația care ne arată dependența între costul total amortizat și costul total real.

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$$

Precedenta relație ne arată că dacă vrem să majorăm costul real folosind costul amortizat atunci $\Phi(D_n) \geqslant \Phi(D_0)$.

În practică nu știm întotdeauna de la început câte operații se vor executa. Dacă impunem ca $\Phi(D_i) \geqslant \Phi(D_0), \forall i \leqslant n$, atunci garantăm că, la fel ca la metoda creditelor, plătim în avans.

De obicei, alegem $\Phi(D_0) = 0$ și doar demonstrăm că $\Phi(D_i) \geqslant 0$

potențial : $\Phi(D_0) = 0$, $\Phi(D_i) \geq 0$
și Φ-urile e.î. să se reducă variabilele din :
$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$

Definiție 4 (Cost amortizat - metoda potențialului). Dacă s-a definit o funcție de potențial, pentru structura de date, care este inițial zero și întotdeauna nenegativ, costul amortizat al unei operații este costul real la care se adaugă diferența de potențial

4.1 O problemă oarecare

Să presupunem că avem de efectuat o succesiune de n operații pe o structură de date în care operația cu numărul i are asociat costul c_i de mai jos.

$$c_i = \begin{cases} i, & \text{dacă i este putere a lui 2} \\ 1, & \text{altfel} \end{cases}$$

Folosiți metoda agregării și/sau metoda creditelor pentru a determina costul amortizat per operație.

• agregare : $\hat{c} = \frac{T(n)}{n}$

$T(n) = 1 + 2 + 1 + 4 + 1 + 1 + 1 + 8 + \dots$

$= n - \log_2 n + \sum_{0}^{\log_2 n} 2^k = n - \log_2 n + \frac{2n-1}{2-1}$

$\hat{c} = \frac{3n - \log_2 n - 1}{n} \simeq 3$

• credite : c_i : 1, 2, 1, 4, 1, 1, 1, 8, 1, 1, ..., 16, 1, 1, ... 32, ...

sparge în grupurile $2^{k-1}+1 \dots 2^k$, a.î. $\sum \hat{c}_i \geq \sum c_i$ pt fiecare grup

fiecare are 2^{k-1} elem $\Rightarrow \sum \hat{c}_i = 2^{k-1} \hat{c}_i$

$\sum c_i = \underbrace{2^{k-1} - 1}_{1\text{-uri}} + \underbrace{2^k}_{2^k} = 2^{k-1} \cdot 3 - 1$

$\sum \hat{c}_i \geq \sum c_i$
$\Leftrightarrow 2^{k-1} \hat{c}_i \geq 2^{k-1} \cdot 3 - 1$
$\Rightarrow \hat{c}_i \simeq 3$ e OK

• potențial

fie $i = 2^{K_i} + R_i$

$\Phi(i) = 2R_i$

$\hat{c}_i = c_i + \Phi(i) - \Phi(i-1)$

dacă $i \neq 2^K \Rightarrow \hat{c}_i = 1 + 2R - (2R-1) = 2$

dacă $i = 2^K \Rightarrow \hat{c}_i = 2^K + 0 - 2(2^{K-1} - 1) = 2$ WOW !!!

$\hat{c}_i = 2$!!! (nu e greșit)

4.2 Stivă cu backup (backup stack)

Să presupunem că avem de efectuat o succesiune de n operații pe o stivă a cărei dimensiune niciodată nu depășește o valoare k. Costul real al acestor operații (push, pop) este constant, si il consideram ca fiind 1. După fiecare k operații, facem o copie (backup) întregii stive (copierea unui element are cost 1 in cadrul acestei operații).

Calculați, folosind metoda creditelor, costul a n operații pe stivă, incluzând operațiile de backup.

Hint: Se poate obține costul total O(n) dacă se aleg costurile amortizate corespunzătoare.

$c_i = \begin{cases} 1 + o_i & , i \mid K \\ 1 & , i \nmid K \end{cases}$, o_i = size

- agregat: cel rapid, luăm $o_i = K$

$T(n) = n + K \cdot \left[\frac{n}{K}\right] \leq 2n$

$\Rightarrow \hat{c} = \frac{T(n)}{n} = 2$

- credite

cel rapid: K push, (K/2 pop, K/2 push) *

luăm $\hat{c}_{push} = 2$ — 1 push, 1 copy

$\hat{c}_{pop} = 2$ — 1 pop, 1 copy pt elem simetric din stivă, care va fi șters

sau $\hat{c}_{pop} = 1$, $\hat{c}_{push} = 3$ — 1 push, 2 copy : $2 \cdot K/2 = K$ = size max

- potențial

$\phi(i) = R$, $i \% K = R$

$\hat{c}_i = c_i + \phi(i) - \phi(i-1)$

dacă $i \nmid K \Rightarrow \hat{c}_i = 1 + R - (R-1) = 2$

dacă $i \mid K \Rightarrow \hat{c}_i = 1 + size_i + 0 - (K-1) = 2 - (K - size_i) \leq 2$ ($K - size_i \geq 0$)

4.3 Vector redimensionabil (resizable array)

Un vector redimensionabil este o structură de date care stochează o secvență de elemente și suportă operațiile de mai jos.

- **AddToEnd(D, x)**: adaugă elementul x la sfârșitul secvenței
- **LookUp(D, k)**: returnează al k-lea element din secvență sau NULL dacă lungimea curentă a secvenței este mai mică decât k

La adaugarea unui element când capacitatea actuală n a vectorului nu are loc pentru elementul respectiv, se rezerva un spatiu nou pentru vector cu capacitate dubla 2n, și se muta elementele existente în primele n locuri ale noului spatiu, cauzând cost n. Apoi se eliberează spațiul folosit anterior de vector. La început presupunem că vectorul are spațiu pentru un element. Dacă există loc în vector presupunem că accesarea locului următor ultimului element și așezarea elementului nou pe acest loc ia timp 1 sau O(1). Care este costul amortizat pentru operația **AddToEnd**?

$c_i = \begin{cases} i, & i = 2^k + 1 \\ 1, & \text{altfel} \end{cases}$

$\log_2(n-1)$

$\log_2(n-1)$

- agregat: $T(n) = n - \log_2(n-1) - 1 + \sum_{0} (2^k + 1)$

$= n + 2(n-1) - 1 = 3n - 3$

$\hat{c} = \frac{T(n)}{n} \leq 3$

- credite

c_i: 1, 2, 3, 1, 5, 1, 1, 1, 9, 1, ..., 17

fiecare grupare $\sum \hat{c}_i \geq \sum c_i$

$\Leftrightarrow 2^k \cdot \hat{c} \geq 2^k - 1 + 2^{k+1} + 1 = 3 \cdot 2^k$

$\Rightarrow \hat{c} \geq 3$ ✓

- potențial

$\Phi(i) = i$ - loc liber

$\hat{c}_i = c_i + \Phi(i) - \Phi(i-1)$

dacă se copiază: $\hat{c}_i = i + (i - (i-1)) - (i - 1 - 0) = 2$

dacă nu, $\hat{c}_i = 1 + (i - \text{vîrful} + 1) - (i - 1 - \text{vîrful} + 1 - 1) = 3$

$\Phi(0) = 0 - 1 = -1$, iar restul $\geq 0 \Rightarrow \Phi(i) \geq \Phi(0)$

4.4 Vector extensibil (extendable array)

Un vector extensibil este o structură de date care stochează o secvență de elemente și suportă operațiile de mai jos.

- **AddToFront(D, x)**: adaugă elementul x la începutul secvenței
- **AddToEnd(D, x)**: adaugă elementul x la sfârșitul secvenței
- **LookUp(D, k)**: returnează al k-lea element din secvență sau NULL dacă lungimea curentă a secvenței este mai mică decât k

Descrieți o simplă structură de date care implementează algoritmul cerut. Primele două operații trebuie să aibă un cost amortizat $O(1)$, operația **LookUp** are worst-case $O(1)$. De asemenea, structura voastră ar trebui să ocupe un spațiu $\theta(n)$, unde n este lungimea curentă a numărului de element din subsecvență.

4.5 O structură de date nouă

Folosind ideea de la Insertion Sort vrem să construim o structură de date care să susțină executarea în mod eficient a operațiilor SEARCH si INSERT.

Operația de SEARCH se execută în $O(\log(n))$ pe un vector sortat (folosing căutare binară). Din păcate, operația INSERT rulează în $O(n)$, întrucât în cel mai defavorabil caz suntem nevoiți să shiftăm toate elementele pentru a face loc pentru cel nou.

Presupunem că dorim să suportăm o secvență de n operații SEARCH și INSERT. Fie $< n_{k-1}n_{k-2}...n_0 >$ reprezentarea binară a lui n (unde $k = [\log(n+1)]$).

În loc să folosim un singur vector sortat de dimensiune n (numit **v**), vom folosi k vectori sortați, numiti $v_0, v_1, ..., v_{k-1}$, care respectă relația următoare.

$$|v_i| = \begin{cases} 2^i, & \text{dacă } n_i = 1 \\ 0, & \text{dacă } n_i = 0 \end{cases}$$

Se observă că, deși fiecare vector este sortat, **nu** există o relație particulară între elementele din vectori diferiți.

Operația de SEARCH se realizează făcând căutare binară.

Operația de INSERT se realizează ca în următorul pseudocod, complexitatea fiind $O(n)$.

```
function Insert(A[1..k], x) {
        // vector care contine doar elementul
        newArray = {x};

        i = 0;
        while (A[i] is NOT empty) {
                // apeleaza subrutina de la Merge Sort
                newArray = merge(A[i], newArray);
                A[i] = empty;
                i++;
        }

        A[i] = newArray;
}
```

Se cere să se determine raspunsul pentru următoarele cerințe:

- Complexitatea operației SEARCH (worst-case).
- Costul amortizat pentru o secvență de n operații de tip INSERT, utilizând metoda potențialului.
- Dați exemple de structuri de date învățate care oferă un timp de rulare mai bun pentru ambele operații.

Hint: $\Phi(D_i) = \sum_{j=0}^{k-1}(k-j)n_j 2^j$

$$K\left(2^{h_{i+1}} - \sum_{2^i=2}^{2^{h_{i+1}-1}} 2^{2^i}\right) = K$$

$$S_n = \sum_{i=1}^{n} i \cdot 2^i = \sum_{i=2}^{n} 2^i + \sum_{n=2}^{n} (i-1)\cdot 2^i = 2^{n+1} - 2 + 2S_{n-1}$$

$$S_n = S_{n-1} + n \cdot 2^n \Rightarrow 2^{n+1} - 2 + 2S_{n-1} = S_{n-1} + n \cdot 2^n$$

$$\Rightarrow S_{n-1} = 2^n(n-2) + 2 \Rightarrow S_n = 2^{n+1}(n-1) + 2$$

$$\epsilon_i = 2^{h_{i-1}+1} - 1 + K + 2^{h_{i-1}}(h_{i-1} - 2) + 2 - h_{i-1}2^{h_{i-1}} =$$

$$= K + 1 \Rightarrow O(\log n)$$

Seminar 7 - Exerciții rezolvate

Echipa AA

02 January 2023

Cuprins

Exercițiul 1 - K Adjacent Equal **1**

Exercițiul 2 - K Path **2**

Exercițiul 3 - Graph Coloring **2**

Exercițiul 4 - K-Clique **2**

Exercițiul 5 - Vertex Cover **3**

Exercițiul 6 - Subset sum problem **3**

Exercițiul 7 - Traveling salesman problem **4**

Exercițiul 8 - Eight queens puzzle **4**

Exercițiul 9 - Subgraph isomorphism problem **5**

Exercițiul 10 - Independent set **6**

Exercițiul 11 - Equal Subset Sum Partition **6**

Exercițiul 12 - SAT (Boolean Satisfiability Problem) **7**

Observație: În redactarea soluțiilor am avut nevoie de structuri de date dincolo de ce oferă limbajul C, dar lipsite de complexitatea celor din C++ sau Java. Cel mai intuitiv ar fi fost să folosim C*, dar din cauza relevanței scăzute în practică a lui, am ales să folosim un limbaj asemănător cu Python 3. În implementare am utilizat Mutable Sequence Types și List Comprehensions. **NU** sunteți obligați să folosiți Python la examen.

Exercițiul 1 - K Adjacent Equal

> Fiind dat un vector de numere, există o subsecvență de elemente egale consecutive de lungime mai mare decât `k`?

```
# V = vectorul, n = nr de elemente din V
def secv(V, n, k):
    # generare
    i = choice(range(n - k))

    # testare
    for j in range(i, i + k):
        if V[j] != V[j + 1]:
            fail()
    success()
```

Încearcă toate pozițiile de start și verifică dacă începe o secvență de `k` elemente egale. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(k)}$$

Exercițiul 2 - K Path

> Având un graf, $G(V, E)$, să se determine dacă există un drum de la nodul `u` la nodul `v` care are lungimea mai mică decât o valoare dată, `dim`.

```
# V = noduri, E = muchii
def path(V, E, u, v, dim):
    # testare
    if u == v:
        success()
    if dim <= 1:
        fail()

    # generare: alegem un vecin al lui u
    k = choice([v for (u, v) in E]) # see list comprehension
    path(G, k, v, dim - 1)
```

Deoarece am ales să stocăm E ca o listă de muchii, complexitatea pentru găsirea vecinilor lui `v` este O(|E|).

Ne alegem un vecin și rezolvăm recursiv până când ajungem la `v` sau avem un `dim` prea mic. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(dim \cdot |E|)}$$

Exercițiul 3 - Graph Coloring

> Dându-se un graf $G(V, E)$ și `k` culori, se pot colora nodurile grafului doar cu cele `k` culori astfel încât niciun nod sa nu aibă un vecin de aceeași culoare?

```
# V = noduri, E = muchii, k = nr culori
def coloring(V, E, k):
    # generare
    for (node in V)
        color[node] = choice(range(k))

    # testare
    for (u, v) in E:
        if color[u] == color[v]:
            fail()
    success()
```

Atribuim o culoare fiecărui nod și testăm dacă avem o colorare validă. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(|V| + |E|)}$$

Exercițiul 4 - K-Clique

> Dându-se un graf $G(V, E)$ și un număr `k`, există un subgraf complet (o clică) de dimensiune `k`?

```
def kclique(V, E, k):
    # generare
    clique = []
    for _ in range(k)
        diff = [v for v in V if v not in clique]
        clique.append(choice(diff))
```

```
    # testare
    for i in clique:
        for j in [v for v in clique if v != i]
            if (i, j) not in E:
                fail()
    success
```

Alegem k noduri, apoi verificăm că formează o clică validă. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(k^2(|V| + |E|))}$$

Exercițiul 5 - Vertex Cover

> Dându-se un graf $G(V, E)$ și un număr k, există o submulțime de k noduri astfel încât fiecare muchie (v_1, v_2) să aibă cel puțin unul dintre nodurile care o compun (v_1 sau v_2) în submulțimea aleasă?

```
def vertex_cover(V, E, k):
    # generare
    cover = []
    for _ in range(k):
        diff = [v for v in V if v not in cover]
        cover.append(choice(diff))

    # testare
    for (i, j) in E:
        if i not in cover or j not in cover:
            fail()
    success()
```

Alegem k noduri, apoi verificăm că formează o acoperire validă. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(k^2|V| + k|E|)}$$

Exercițiul 6 - Subset sum problem

> Se dă o mulțime de N numere și un număr Q. Există o submulțime de numere a căror sumă să fie fix Q?

```
# M = multimea
def subset_sum(M, Q):
    # generare
    sum = 0
    for x in M:
        if choice([True, False]):
            sum = sum + x

    # testare
    if sum == Q:
        success()
    else:
        fail
```

Pentru fiecare element din mulțime alegem dacă să îl folosim în sumă sau nu, obținând toate sumele posibile. Dacă există o submulțime care satisface cerința, ramura corespunzătoare ei din arborele nedeterminist va întoarce succes. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(|M|) = O(N)}$$

Exercițiul 7 - Traveling salesman problem

> Se dă o mulțime de orașe conectate între ele prin drumuri. Există vreo modalitate ca un comis voiajor să viziteze toate orașele o singură dată și să se întoarcă de unde a plecat?

```
# V = noduri, E = muchii
def TSP(V, E):
    # generare
    tour = []
    for _ in V:
        diff = [v for v in V if v not in tour]
        tour.append(choice(diff))
    tour.append(tour[0])

    # testare
    for i in range(len(V)):
        if (tour[i], tour[i + 1]) not in E:
            fail()
    success()
```

Generăm o permutare de noduri, apoi verificăm că aceasta formează un drum în graf. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(|V|^2 + |E|)}$$

Exercițiul 8 - Eight queens puzzle

> Plasați 8 regine pe o tablă de șah fără ca acestea să se atace.

```
def queens():
    row = 8 * [False]
    diagP = 15 * [False]
    diagS = 15 * [False]

    for x in range(8):
        # generare
        y = choice(range(8))

        # testare
        if row[y] or diagP[x - y + 7] or diagS[x + y]:
            fail()

        row[y] = diagP[x - y + 7] = diagS[x + y] = True

    success()
```

Alegem o poziție pentru fiecare regină, apoi verificăm dacă acea linie sau coloană sau diagonală este deja ocupată. Complexitatea temporală a algoritmului nedeterminist este, unde am notat cu n dimensiunea tablei de șah:

$$\boxed{O(N)}$$

O altă soluție, recursivă, ar fi:

```
def queensRec(currentQueens):
    if len(currentQueens) == 8:
        success()
```

```
    x, y = choice(range(8)), choice(range(8))

    for (qx, qy) in currentQueens:
        if x == qx or y == qy or x - y == qx - qy or x + y == qx + qy:
            fail()

    queensRec(currentQueens + [(x, y)])

def queens():
    queensRec([])
```

Această soluție seamănă cu cea clasică, de backtracking. Deoarece verificăm reginele între ele, complexitatea este:

$$\boxed{O(N^2)}$$

Exercițiul 9 - Subgraph isomorphism problem

> Două grafuri $G_1(V_1, E_1)$ și $G_2(V_2, E_2)$ sunt izomorfe dacă există o funcție bijectivă $f : V_1 \to V_2$, astfel încat muchia $(u, v) \in E_1 \iff$ muchia $(f(u), f(v)) \in E_2$. (Observație: doua grafuri pot fi izomorfe doar dacă au același număr de noduri.)

Dându-se două grafuri, G_1 și G_2, există un subgraf în G_1 care să fie izomorf cu G_2?

```
def subgraph_isomorphism(V1, E1, V2, E2):
    f = {} # dictionar pentru functia f
    ff = {} # f^-1

    #  generare
    available = V1
    for v2 in V2:
        v1 = choice(available)
        f[v1] = v2
        ff[v2] = v1
        available.remove(v1)

    # testare
    for (u, v) in E2:
        if (ff[u], ff[v]) not in E1:
            fail()

    # V1' = f.keys()
    # E1' = [(ff[u], ff[v]) for (u, v) in E2]

    success()
```

Pentru subgraful lui G_1, trebuie să alegem atât nodurile, cât și muchiile. Funcția este bijectiva, așa că putem privi dinspre G_2: alegem un $v_1 \in V_1$ pentru fiecare $v_2 \in V_2$, apoi verificăm că toate muchiile din E_2 au corespondent în E_1. Toate muchiile în plus din E_1 sunt ignorate.

Pentru generare avem complexitatea $O(|V_1| \cdot |V_2|)$, din cauza faptului că acel remove are $O(|V_1|)$. Pentru testare avem $O(|E_1| \cdot |E_2|)$. În total, complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(|V_1| \cdot |V_2| + |E_1| \cdot |E_2|)}$$

Exercițiul 10 - Independent set

> Dându-se un graf $G(V, E)$ și un număr `k`, există o mulțime S de `k` noduri astfel încât orice muchie are cel mult un capăt în S?

```
def independent(V, E, k):
    # generare
    S = []
    for _ in range(k):
        diff = [v for v in V if v not in S]
        S.append(choice(diff))

    # testare
    for (u, v) in E:
        if u in S and v in S:
            fail()
    success()
```

Ne alegem o submulțime de `k` noduri și verificăm condiția. Complexitatea temporală a algoritmului nedeterminist est:

$$\boxed{O(k^2|V| + k|E|)}$$

Exercițiul 11 - Equal Subset Sum Partition

> Dându-se o mulțime de n numere întregi, există o împărțire a elementelor sale în două submulțimi S_1 și S_2 care sa aibă sume egale?

```
# M = multimea
def partition(M):
    s = 0 # suma totala
    s1 = 0 # suma pentru S1

    # generare
    for x in M:
        s = s + x
        if choice([True, False]):
            s1 = s1 + x

    # testare
    if s == 2 * s1:
        success()
    else
        fail()
```

Îmi aleg submulțimea S_1. Din moment ce $\sum S_1 = \sum S_2$, verificăm că

$$2 \cdot s_1 = s_1 + s_2 = s$$

Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(n)}$$

Exercițiul 12 - SAT (Boolean Satisfiability Problem)

> Se dă o expresie booleană în forma normală conjunctivă - o conjuncție de clauze, unde clauzele sunt disjuncții. Exemplu:
>
> $$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_1$$
>
> Să se determine daca exista o posibilitate de atribuire a variabilelor astfel incat expresia sa fie adevarata.

```python
# V = variabile, C = clauze
def sat(V, C):
    A = {} # dicționar de atribuiri pentru fiecare variabilă

    # generare
    for v in V:
        A[v] = choice([True, False])
        A[-v] = not A[v]

    # testare
    for clause in C:
        if not any(A[v] for v in clause):
            fail()
    success()
```

Atribui fiecărei variabile o valoare de adevăr și verific dacă satisfac toate clauzele. Complexitatea temporală a algoritmului nedeterminist este:

$$\boxed{O(|V| \cdot |C|)}$$

Seminar 8 - Exerciții rezolvate

Echipa AA

06 January 2023

Cuprins

Exercițiul 1 **1**

Exercițiul 2 **2**

Exercițiul 3 **3**

Exercițiul 4 **3**

Bibliografie **4**

Exercițiul 1

> **Problema k-Colorare** ($k \geq 2$): Dându-se un graf neorientat $G = (V, E)$ și k culori, se pot colora nodurile grafului folosind cele k culori astfel încât niciun nod sa nu aibă un vecin de aceeași culoare?

Demonstrați următoare reducere polinomială:

$$\text{3-Colorare} \leq_p \text{4-Colorare}$$

Inputul pentru ambele probleme este un graf, deci:

$$F : G \to G$$

La trecerea de la 3-Colorare la 4-Colorare, adaugăm un nod nou, unit cu toate celelalte noduri.

```python
def F(G: tuple):
    V, E = G
    # V conține noduri numerotate de la 0 la len(V) - 1
    # E conține muchiile, de tip (nod1, nod2)

    last = len(V) # adăugăm noul nod
    V.append(last)
    E = E + [(v, last) for v in V] # unim nodul cu toate muchiile

    return (V, E)
```

Complexitatea temporală a lui `F` este liniară.
Pentru a demonstra reducerea:

$$\text{3-Colorare} \leq_p \text{4-Colorare}$$

Trebuie să verificăm echivalența următoare conform ideii din referința [1]:

$$\text{3-Colorare}(G) = 1 \iff \text{4-Colorare}(F(G)) = 1$$

Analizăm implicația directă:

$$\text{3-Colorare}(G) = 1 \Rightarrow \text{4-Colorare}(F(G)) = 1$$

Pentru orice situație în care graful se poate colora cu 3 culori, nodul nou va trebui colorat cu o singură *altă* culoare, deci va respecta 4-Colorare.

Analizăm implicația inversă:

$$\text{3-Colorare}(G) = 1 \Leftarrow \text{4-Colorare}(F(G)) = 1$$

Avem un nod (cel adaugat de noi, numerotat cu $|V| + 1$) care este conectat la toate celelalte noduri, deci trebuie sa aibă o culoare diferită față de toate. Pentru ca problema este 4-Colorabilă, înseamnă că, dacă scoatem acest nod, graful care rămâne (cel original) este 3-Colorabil.

Exercițiul 2

> **Problema Partiției**: Dându-se un număr n și o mulțime de n întregi $A = \{a_1, ..., a_n\}$, există $S \subset \{1, \ldots, n\}$ astfel încât $\sum_{i \in S} a_i = \sum_{i \notin S} a_i$?
>
> **Problema sumei de submulțime, q-Sume (Subset Sum)**: Dându-se numerele t, q și o mulțime de t întregi $B = \{b_1, \ldots, b_t\}$, există $S' \subseteq \{1, \ldots, t\}$ astfel încât $\sum_{i \in S'} b_i = q$?
>
> Demonstrați următoare reducere polinomială:
>
> $$\text{Partiție} \leq_p q\text{-Sume}$$

Intrarea pentru problema Partiției este o mulțime, iar pentru q-Sume este o mulțime și un întreg, deci:

$$F : Set \to (Set, Int)$$

```
def F(A: set):
    q = sum(A) / 2
    B = A

    return B, q
```

Complexitatea temporală a lui `F` este liniară.

Pentru a demonstra reducerea trebuie să verificăm echivalența următoare conform ideii din referința [2]:

$$\text{Partiție}(A) = 1 \iff q\text{-Sume}(\underbrace{F(A)}_{B}) = 1$$

Analizăm implicația directă:

$$\text{Partiție}(A) = 1 \Rightarrow q\text{-Sume}(\underbrace{F(A)}_{B}) = 1$$

Dacă problema partiției are soluție pentru intrarea A, înseamnă că există o submulțime de indici S astfel încat

$$\sum_{i \in S} a_i = \sum_{i \notin S} a_i = \frac{1}{2} \sum A$$

Pentru că l-am fixat pe `q` să fie `sum(A) / 2`, înseamnă că submulțimea de indici S respectă condiția din q-Sume.

Analizăm implicația inversă:

$$\text{Partiție}(A) = 1 \Leftarrow q\text{-Sume}(\underbrace{F(A)}_{B}) = 1$$

Problema q-Sume este satisfacută, deci există o submulțime de indici S' a lui B, astfel încât

$$\sum_{i \in S'} b_i = q$$

Având în vedere că $B = A$, iar $q = \frac{1}{2} \sum A$

$$\sum_{i \in S'} a_i = \frac{1}{2} \sum A$$

Înseamnă că ce rămâne în afara lui S', $\{a_i | i \notin S'\}$, va însuma tot la `sum(A) / 2`, deci există două submulțimi disjuncte și complementare care să aibă aceeași sumă, respectând condiția de la Partiție.

Exercițiul 3

> **Problema Vertex Cover**: Dându-se un graf neorientat $G' = (V', E')$ și un număr k', există o submulțime S' de cel mult k' noduri astfel încât fiecare muchie are *cel puțin* un capăt în S'?

> **Problema Independent Set**: Dându-se un graf neorientat $G = (V, E)$ și un număr k, există o submulțime S de cel puțin k noduri, astfel încât fiecare muchie are *cel mult* un capăt în S?

Demonstrați următoare reducere polinomială:

$$\text{Independent-Set} \leq_p \text{Vertex-Cover}$$

Inputul pentru ambele este un graf și un întreg, deci:

$$F : (\underbrace{(Set, List)}_{Graph}, Int) \to (Graph, Int)$$

```
def F(G: tuple, k: int):
    V, _ = G
    return G, len(V) - k
```

Complexitatea temporală a lui `F` este liniară.
Pentru a demonstra reducerea trebuie să verificăm echivalența următoare conform ideii din referința [3]:

$$\text{Independent-Set}(G, k) = 1 \iff \text{Vertex-Cover}(\underbrace{F(G, k)}_{(G', k')}) = 1$$

Observație: $G' = G, V = V', E = E'$.
Analizăm implicația directă:

$$\text{Independent-Set}(G, k) = 1 \Rightarrow \text{Vertex-Cover}(\underbrace{F(G, k)}_{(G', k')}) = 1$$

Dacă Independent-Set poate fi satisfacută, înseamnă că oricare latură $e = (u, v) \in E$ are *cel mult* un capăt într-o submulțime de cel puțin k noduri S. Ceea ce înseamnă că are *cel puțin* un capăt în $V \setminus S = S'$. $k' = |V| - k \wedge |S| \leq k \Rightarrow |S'| \geq k'$, deci graful respectă condiția de la Vertex-Cover pentru k'.
Analizăm implicația inversă:

$$\text{Independent-Set}(G, k) = 1 \Leftarrow \text{Vertex-Cover}(\underbrace{F(G, k)}_{(G', k')}) = 1$$

Dacă Vertex-Cover poate fi satisfacută, înseamnă că oricare latură $e = (u, v) \in E$ are *cel puțin* un capăt într-o submulțime de cel mult k' noduri S'. Ceea ce înseamnă că are *cel mult* un capăt în $V \setminus S' = S$. $k' = |V| - k \wedge |S'| \geq k' \Rightarrow |S| \leq k$, deci graful respectă condiția de la Independent-Set pentru k.

Exercițiul 4

> *Problema Set Cover*: Dându-se o mulțime U, o colecție $S_1, S_2, \ldots, S_m$ de submulțimi ale lui U și un număr k', există o colecție de cel mult k' astfel de submulțimi care reunite să dea U?

Demonstrați următoare reducere polinomială:

$$\text{Vertex-Cover} \leq_p \text{Set-Cover}$$

Inputul pentru Vertex-Cover este un graf și un întreg, iar pentru Set-Cover este o listă, o listă de liste și un întreg.

$$F : (\underbrace{(Set, List)}_{Graph}, Int) \to (Set, List < Set >, int)$$

```
def F(G, k: int)
    V, E = G

    U = E
    S = []
    for _ in V:
        S.append(set())

    for i in V:
        for u, v in E:
            if u == i or v == i:
                S[i].add((u, v))

    return (E, S, k)
```

Complexitatea temporală a lui `F` este $O(|V| \cdot |E|)$, considerăm inserția într-o mulțime $O(1)$.
Pentru a demonstra reducerea trebuie să verificăm echivalența următoare conform ideii din referința [4]:

$$\text{Vertex-Cover}(G, k) = 1 \iff \text{Set-Cover}(\underbrace{F(G, k)}_{(U,S,k')}) = 1$$

Observație: $U = E, k = k'$.
Analizăm implicația directă:

$$\text{Vertex-Cover}(G, k) = 1 \Rightarrow \text{Set-Cover}(\underbrace{F(G, k)}_{(U,S,k')}) = 1$$

Dacă Vertex-Cover este satisfacută, fiecare muchie din G are cel puțin un nod în mulțimea de acoperire S'. Din construcția lui `F`, vedem că muchiile nodurilor din această mulțime sunt suficiente pentru a acoperi tot graful. Pentru că mulțimea U din problema Set-Cover este mulțimea muchiilor din graf, E, înseamnă că există o submulțime de mulțimi din S, de dimensiune mai mică de k' care acoperă întreaga listă.
Analizăm implicația inversă:

$$\text{Vertex-Cover}(G, k) = 1 \Leftarrow \text{Set-Cover}(\underbrace{F(G, k)}_{(U,S,k')}) = 1$$

Invers, dacă Set-Cover este rezolvabilă, înseamnă că există o submulțime de mulțimi din S care, reunite, acoperă toată mulțimea U. Din construcția lui `F`, fiecare index din această submulțime este un nod în V. Mulțimea acestor noduri S' conține noduri de pe fiecare muchie (prin definiția lui Set-Cover), deci respectă condiția de Vertex-Cover.

Bibliografie

[1] „Graph Theory - How to Prove That the 4-Coloring Problem Is NP-complete - Mathematics Stack Exchange". Disponibil la: https://math.stackexchange.com/questions/3241339/how-to-prove-that-the-4-coloring-problem-is-np-complete

[2] „Complexity Theory - Karp Reduction from PARTITION to SUBSET SUM - Stack Overflow". Disponibil la: https://stackoverflow.com/questions/42063810/karp-reduction-from-partition-to-subset-sum

[3] „28.17. Reduction of Independent Set to Vertex Cover — OpenDSA Data Structures and Algorithms Modules Collection". Disponibil la: https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/independentSet/_to/_vertexCover.html

[4] „Introduction to Algorithms Proving NP-Completeness CS 482 Spring 2006". 2006. Disponibil la: http://www.cs.cornell.edu/courses/cs482/2007sp/NPComplete.pdf

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, și C. Stein, „Introduction to Algorithms. third", *New York*, 2009.

1.1 Demonstrarea corectitudinii [1]

Pentru a demonstra **corectitudinea**, trebuie să luăm în calcul acest "orice". Demonstraţia corectitudinii se face în **3 paşi**:

1. Identificarea precondiţiilor algoritmului. Totalitatea precondiţiilor formează **aserţiunea de intrare** notată $P_I : I \to \{0, 1\}$, unde $I =$ *domeniul datelor de intrare.*
2. Identificare postcondiţiilor algoritmului. Totalitatea postcondiţiilor formează **aserţiunea de ieşire** notată $P_O : I \times O \to \{0, 1\}$, unde $O =$ *domeniul rezultatelor*
3. Demonstraţia că, plecând din starea iniţială a algoritmului, în care se respectă aserţiunea de intrare, paşii descrişi de algoritm ne duc într-o stare finală în care se respectă aserţiunea de ieşire.

Ideea demonstraţiei este să identificăm forma generală a stărilor intermediare prin care ar trebui să treacă algoritmul şi să ne asigurăm că la fiecare acţiune algoritmul trece dintr-o stare validă în altă stare validă.

2.3.2 Sortarea prin selecție

```
1:  function SelectionSort(V[0..n − 1])
2:      for i = 0; i < n − 1; i + + do
3:          min = i
4:          for j = i + 1; j < n; j + + do
5:              if v[j] < v[min] then
6:                  min = j
7:              end if
8:          end for
9:          swap(v[i], v[min])
10:     end for
11:     return V
12: end function
```

1. **Invariant pentru bucla interioară:** Înainte de iteraţia j:
   min_j = indexul elementului minim din $v[i..j-1]$

   (a) **Iniţializare:** Înainte de prima iteraţie, $min_{i+1} = i =$ indexul elementului minim din $v[i..i]$

   (b) **Menţinere:** Presupunem că, înainte de iteraţia j,
   min_j = indexul elementului minim din $v[i..j-1]$
   După iteraţia j (înainte de iteraţia j + 1), dacă $v[j] < v[min_j]$, atunci $min_{j+1} = j$, altfel $min_{j+1} = min_j$.
   Aşadar, min_{j+1} = indexul elementului minim din $v[i..j]$

   (c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $j = n$):
   min_n = indexul elementului minim din $v[i..n-1]$

2. **Invariant pentru bucla exterioară:** Înainte de iteraţia i, $v[0..i-1]$ este sortat **şi conţine cele mai mici i elemente din v**.

   (a) **Iniţializare:** Înainte de prima iteraţie, $v[0.. - 1]$ este sortat şi conţine cele mai mici 0 elemente din v.

   (b) **Menţinere:** Presupunem că, înainte de iteraţia i, $v[0..i-1]$ este sortat şi conţine cele mai mici i elemente din v.
   După iteraţia i (înainte de iteraţia i + 1), $v[0..i-1]$ rămâne nemodificat, iar $v[i]$ primeşte valoarea $v[min]$, unde $min =$ indexul elementului minim din $v[i..n-1]$ (conform invariantului anterior).

   Aşadar, înainte de iteraţia i + 1, $v[0..i]$ sortat şi conţine cele mai mici i + 1 elemente din v.

(c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $i = n - 1$): $v[0..n-2]$ este sortat şi conţine cele mai mici n - 1 elemente din v. Rezultă că $v[n-1]$ conţine cel mai mare element din v, deci $v[0..n-1]$ sortat.

Prin invarianții selectați am demonstrat că funcția SelectionSort sortează vectorul V.

3 Exerciţii

3.1 Căutarea binară [2]

```
1: function BinarySearch(V, x, start, stop)
2:     while start ≤ stop do
3:         mid = (start + stop)/2
4:         if (x == v[mid]) then return mid
5:         end if
6:         if (x > v[mid]) then
7:             start = mid + 1
8:         else
9:             stop = mid − 1
10:        end if
11:    end while
12:    return Null
13: end function
```

np $x \in V$

invariant: $x \in V[start \dots stop]$

I $x \in V$, start = 0, stop = n-1
$\Rightarrow x \in V[start \dots stop]$

II np $x \in V[start, stop]$

dacă $x = mid \Rightarrow$ l-am găsit

dacă $x < mid \Rightarrow x < y, \forall y \in V[mid \dots stop]$
V sortat
$\Rightarrow x \in V[start \dots mid-1]$

acem pt $x > mid$
$\Rightarrow x \in V[start' \dots stop']$

III: când ajungem la start = stop
$\Rightarrow x \in V[start \dots start] \Rightarrow x = V[start]$
$\Rightarrow$ l-am găsit sigur

dacă $x \notin V \Rightarrow$ întoarce null
intervalul se micşorează la fiecare pas $\Rightarrow$ se termină

3.2 Sortarea prin inserţie

```
1: function InsertionSort(V[0..n − 1])
2:     for j = 1; j < n; j++ do
3:         x = v[j]
4:         i = j − 1
5:         while i >= 0 and v[i] > x do
6:             v[i + 1] = v[i]
7:             i − −
8:         end while
9:         v[i + 1] = x
10:    end for
11: end function
```

invariant lucrăm interioară:

$x \in \min(V[i+2 \dots j])$

$v[i] \text{ final} \geq x$

I $i = j-1$; $\min(V[i+1 \dots j])$ nu există

II pp că înainte de iterația i, $x \in \min(V[i+2 \dots j])$

după, dacă $v[i] > x \Rightarrow v[i+1] = v[i]$

$\Rightarrow x \in \min(V[i+1 \dots j])$

$\Rightarrow x \in$ ok

↳ dacă $v[i] \leq x \Rightarrow$ nu mai face

III dacă $i = -1 \Rightarrow v[0] \geq x$, $x \in \min(V[1, 2])$

dacă $v[i] \leq x \Rightarrow v[i] \leq v[i+1] = x \in \min(V[i+2, j])$

invariant lucrăm ext: $V[0 \dots j-1]$ sortat

I $j = 1$; $V[0 \dots 0]$ sortat

II înainte de j, pp $V[0 \dots j-1]$ sortat

după lucrăm interioară, $v[i] \leq v[i+1] \leq \min(v[i+2 \dots j])$

sortat ← $V[0 \dots i, i+2 \dots j] = v[0 \dots j-1]$ de dinainte

- pp $i+1$ l-am inserat pe locul $v[j]$

$\Rightarrow v[0 \dots j]$ sortat

III la final, $j = n$, $v[0 \dots n-1]$ sortat

3.3 Sortarea prin metoda bulelor

```
1: function BubbleSort(V[0..n − 1])
2:     for i = 0; j < n; j + + do
3:         for j = n − 1; j > i; j − − do
4:             if v[j] < v[j − 1] then
5:                 swap(v[j], v[j − 1])
6:             end if
7:         end for
8:     end for
9: end function
```

inv int: $V[j] = \min V[j..n-1]$

inv ext: $V[0..i-1]$ sortat cu cele mai mici elem ale lui V

• int: I: $j = n-1$; $V[j..n-1] =$

II pp că înainte de iterația j, $V[j] = \min V[j..n-1]$

după, dacă $v[j] < v[j-1] \Rightarrow$ swap $(v[j], v[j-1])$

$\Rightarrow v[j-1] < v[j]$, iar $V[j] = \min V[j..n-1]$

$\Rightarrow v[j-1] = \min V[j-1..n-1]$

↳ dacă $v[j] \geq v[j-1]$, avem $V[j] = \min V[j..n-1]$

$\Rightarrow v[j-1] = \min V[j-1..n-1]$

III final: $j = i$, $V[i] = \min V[i..n-1]$

• ext: I $i = 0$, $V[0..-1]$ nu ∃

I pp că înainte de iterația i, $V[0..i-1]$ sortat cu cele mai mici el din V

după bucla int, $V[i] = \min V[i..n-1]$ $\Rightarrow V[0..i]$ are cele mai mici el

$V[0..i-1]$ are cele mai mici elem

↳ $V[0..i]$ sortat

III la final, $V[0..n-1]$ sortat (adică tot V)

3.4 Funcţia de partiţionare din cadrul algoritmului QuickSort [3]

```
1: function PARTITION(V, start, stop)
2:     x = v[stop]
3:     i = start − 1
4:     for j = start; j < stop; ++j do
5:         if v[j] < x then
6:             ++i
7:             swap(v[i], v[j])
8:         end if
9:     end for
10:    swap(v[i + 1], v[stop])
11:    return i + 1
12: end function
```

inv: $\forall y < x, \forall y \in V[start \dots i]$

nr de $y, y < x, y \notin V[start, \dots j-1]$ este $i - start + 1$

I $V[start \dots i]$ și $V[start \dots j-1]$ sunt egale

II înainte de j, pt inv

dacă $V[j] < x \Rightarrow$ pt de $y < x$ se incrementează ✓

(swap $\Rightarrow v[i+1] < x$ și $x > v[start \dots i]$)

$\Rightarrow$ ține

III le final $x > V[start \dots i]$

nr de $y < x = i - start + 1 = |V[start, \dots i]$

$\Rightarrow V[start \dots i] < x \leq V[i+1 \dots stop]$

dacă îl pun pe x pe $i+1$, avem (swap ok)

$V[start \dots i] < V[i+1] \leq V[i+2 \dots stop]$

$\Rightarrow$ partiționat la $i+1$

3.5 Funcţia de interclasare din cadrul algoritmului MergeSort

```
1: function MERGE(A, p, q, r)
2:     n1 = q − p + 1
3:     n2 = r − q
4:     for i = 1; i <= n1; ++i do
5:         L[i] = A[p + i − 1]
6:     end for
7:     L[n1 + 1] = ∞
8:     for i = 1; i <= n2; ++i do
9:         R[i] = A[q + i]
10:    end for
11:    R[n2 + 1] = ∞
12:    i = 1
13:    j = 1
14:    for k = p; k ≤ r; ++k do
15:        if L[i] <= R[j] then
16:            A[k] = L[i]
17:            i = i + 1
18:        else
19:            A[k] = R[j]
20:            j = j + 1
21:        end if
22:    end for
23: end function
```

inv ultimului for:

A[p...k-1] sortat

A[p...k-1] conține toate elem din L[1...i-1] și R[1...j-1]

p. L și R sortate

I K=p ⇒ A[p...p-1] nu ∃

II p.p

dacă L[i] ≤ R[j], A[k] = L[i++]

L și A[p...k-1] sortate

dacă la pasul anterior am luat din R ⇒ L[i] > R[j-1] = A[k-1]

⇒ L[i] > A[k] ⇒ A[p...k] sortat

↪ dacă am luat din L, știm că L sortat ⇒ L[i] > L[i-1] = A[k-1]

⇒ ↓

↪ dacă L[i] > R[j], A[k] = R[j++]

analog L

p.p.2 se respectă, pt că la fiecare pas iese un el și inc indicele

III la final, A[p...r] sortat și are toate el din L și R (fără ∞-uri)

der L = A[p...q] și R = A[q+1...r]

⇒ A[p...r] conține elementele inițiale, sortate

3.6 Determinarea distanţei minime prin algoritmul lui Dijkstra

```
1: function DIJKSTRA(graph, c, s, adj)
2:     for each v ∈ vertices(graph) do
3:         d[i] = ∞
4:     end for
5:     d[s] = 0
6:     q = priority_queue(graph)
7:     while q do
8:         u = extractMin(q)
9:         q.remove(u)
10:        for each v ∈ adj[u] do
11:            if d[u] + c[u, v] < d[v] then
12:                d[v] = d[u] + c[u, v]
13:            end if
14:        end for
15:    end while
16: end function
```

fie proc mulţimea nodurilor procesate complet (scoase din coadă) = graf \ z

inv: d[proc] min ext

d[proc] conţine cele mai mici el din d final

I proc gol

II pp d[proc] min şi conţine cele mai mici el din d final

u = min(d[u ∈ z])

d[u] ≥ d[x], ∀ x ∈ proc

fie u', drum min u = drum min u' + u' → u

c[u', u] ≥ 0 ⇒ d[u] ≥ d[u']

cum ↑, iar d[proc] conţine cele mai mici el din d fin ⇒ u' ∈ proc

⇒ s-a rulat linia 12 pentru (u', u) ⇒ d[u] min

pp prin absurd că d[proc ∪ {u}] nu conţine el min din d final

⇒ ∃ t ∈ z, d_fin[t] < d[u]

dacă t' (pred lui t în drumul optim) ∈ proc

⇒ d[t] deja înbunătăţit ⇒ nu s-ar fi scos u ∈ z cu d[u] min ↯

altfel, t' ∈ z : c[t', t] ≥ 0 ⇒ d[t] = d[t'] + c[u, t] ≥ d[u]

d[u] min în z ⇒ ≥ d[u]

III la final z gol, proc = graf ⇒ d min pt toate nodurile

dem e varză :) prof era zeu de un inv int cu care să demonstrăm că dacă u e predecesorul lui v în drumul optim până la v, atunci d[v] e optim ceea ce spune acest IDK

1. O expresie aritmetică complet parantezată este definită astfel:
$0, 1, x, [e1 + e2], [e1 * e2], [-e2]$
3 constructori nulari (primii 3 enumerați mai sus), 3 constructori interni (ultimii 3 enumerați mai sus).
Să notăm cu E acest tip de date.
Se definesc operatorii:
***eval*(e, n) : E * N → N**

- (E1) $eval(0, n) = 0$
- (E2) $eval(1, n) = 1$
- (E3) $eval(x, n) = n$
- (E4) $eval([e1 + e2], n) = eval(e1, n) + eval(e2, n)$
- (E5) $eval([e1 * e2], n) = eval(e1, n) * eval(e2, n)$
- (E6) $eval([-e1], n) = -eval(e1, n)$

***subst*(e, f) : E * E → E**

- (S1) $subst(0, f) = 0$
- (S2) $subst(1, f) = 1$
- (S3) $subst(x, f) = f$
- (S4) $subst([e1 + e2], f) = [subst(e1, f) + subst(e2, f)]$
- (S5) $subst([e1 * e2], f) = [subst(e1, f) * subst(e2, f)]$
- (S6) $subst([-e], f) = [-subst(e, f)]$

Operatorul eval(e, n) substituie în expresia e fiecare apariție a lui x cu n și întoarce rezulatul ei.
Operatorul subst(e, f) substituie în expresie e fiecare apariție a lui x cu f și întoarce noua expresie formată.
Să se demonstreze prin inductie structurala ca, pentru orice e, f ∈ E și n ∈ ℕ, proprietatea urmatoare este adevarata:
eval(subst(e, f), n)= eval(e, eval(f, n))

Inițializare

luăm toți constructorii nulari (și externi):
0, 1, x

e = 0
? eval(subst(0, f), n) = eval(0, eval(f, n))
(S1), (E1) (=) eval(0, n) = 0 (E1) (=) 0 = 0 ✓

e = 1
? eval(subst(1, f), n) = eval(1, eval(f, n))
(S2), (E2) (=) eval(1, n) = 1 (E2) (=) 1 = 1 ✓

e = x
? eval(subst(x, f), n) = eval(x, eval(f, n))
(S3), (E3) (=) eval(f, n) = eval(f, n) ✓

Ip. inducție:
știm: eval(subst(e1, f), n) = eval(e1, eval(f, n)) (I1)
și eval(subst(e2, f), n) = eval(e2, eval(f, n)) (I2)

vrem:
A) eval(subst([e1+e2], f), n) = eval([e1+e2], eval(f, n))
B) eval(subst([e1*e2], f), n) = eval([e1*e2], eval(f, n))
C) eval(subst([-e1], f), n) = eval([-e2], eval(f, n))

A) (S4), (E4) (=) eval([subst(e1, f) + subst(e2, f), n] =
= eval(e1, eval(f, n)) + eval(e1, eval(f, n))
(E4) (=) eval(subst(e1, f), n) + eval(subst(e2, f), n) = ...
(I1), (I2) (=) eval(e1, eval(f, n)) + eval(e1, eval(f, n)) = ... ✓

idem pt B

c) (SG)(EG) (=) eval([-subst(e1,f)],n) = -eval(e1, eval(f,n))

(EG) (=) -eval(subst(e2=f, e2), n) = eval(e1, eval(f, n))

cele 2 in ce ~ ✓

=) YEY !!!

...

2. Fie tipul de date T LIST, definit prin constructorii de bază:

- [] :→ $TLIST$
- : : $T * TLIST \to TLIST$

Se consideră **operatorii**:

head(l) : TLIST\[] → T

M1 • head(a:l) = a

range(l) : TLIST → BOOL

R1 • $range([]) = false$

R2 • $range(a : []) = (a == 1)$

R3 • $range(a : l) = (a - 1 == head(l))\&\&range(l)$, pentru $l != []$

sum(l) : TLIST → ℕ

S1 • $sum([]) = 0$

S2 • $sum(a : l) = a + sum(l)$

Demonstrați următoarea proprietate prin inducție structurală:

!!***range(l) → sum(l) == head(l) * (head(l) + 1)/2***

I cea horă : [ ]

? range([ ]) → sum([ ]) == head([ ]) * (head([ ]) + 1) / 2

(=) (R1) fals → ceva

=) adevărat mereu

II inducție:

(!!)

știm: range(l) → sum(l) == ...

vrem: ? range(a:l) → sum(a:l) == head(a:l) * (head(a:l)+1)/2

(S2, M1)

(=) range(a:l) → a + sum(l) == a * (a+1)/2

dacă l = [ ]

(R2) (=) (a == 1) → a + sum([ ]) == a * (a+1)/2

(S1) (=) (a == 1) → a == a * (a+1)/2

dacă a = 1 => true → 1 == 1 * 2 = 2 ✓

dacă a ≠ 1 => fals → ceva ✓

deca $l \neq []$

R3
$(=) (a - 1 == head(l))\&\&range(l).$ → $a + sum(l) == a \cdot (a+1)/2$

deca $a-1 \neq head(l)$: false → cera ✓
deca $a-1 == head(l)$:
⇒ $a = head(l)+1$ /2
→ $(=)$ $range(l)$ → $head(l)+1+sum(l) == (head(l)+1)\cdot(head(l)+2)$
$(head(l)+1)\cdot(head(l)+2)/2 = (head(l)+1)\cdot head(l)/2 + (head(l)+1)\cdot 2/2$
$(=)$ $range(l)$ → $sum(l) == (head(l)+1)\cdot head(l)/2$
↳ l7 ✓

⇒ URAA!!!

3. Fie tipul de date LIST care reprezintă mulțimea listelor cu numere întregi definit prin următorii constructori de bază:
   - $[] :\rightarrow LIST$
   - $(x : l) : INT * LIST \rightarrow LIST$

Fie FUN mulțimea funcțiilor definite astfel: $INT * INT \rightarrow INT$.
Se definesc funcțiile (operatori) următoare, cu axiomele aferente:

fold* : *FUN* * *INT* * *LIST* → *INT
F1 • $fold(f, z, []) = z$
F2 • $fold(f, z, x : l) = f(x, fold(f, z, l))$

len* : *LIST* → *INT
L1 • $len([]) = 0$
L2 • $len(x : l) = 1 + len(l)$

Să se demonstreze prin inducție structurală că P(l) este adevarată, pentru ∀ l ∈ LIST:
L1 **$P(l) = (len(l) = fold(inc, 0, l))$**
cu inc definită astfel $inc(x, y) = y + 1$ 1'1

I cea de bază: $[]$

$P([]) = (len([]) = fold(inc, 0, []))$
L1, F1
$(=)$ $0 = 0$ ✓

II inducție:
știm: $len(l) = fold(inc, 0, l)$
vrem: $len(x:l) = fold(inc, 0, x:l)$

L2, F2
$(=)$ $1 + len(l) = inc(x, fold(inc, 0, l))$
1'1
$(=)$ $1 + len(l) = fold(inc, 0, l) + 1$ $(=)$ l1 ✓

⇒ !!!!

4. Fie tipul de date asbtract BinTree definit prin constructorii de bază:

empty : $\rightarrow BinTree$ # Arborele binar vid

node : $BinTree * Elem * BinTree \rightarrow BinTree$ # Subarborele stâng, rădăcina, sub drept

Se consideră operatorii:

size **:** $\boldsymbol{BinTree \rightarrow N}$ # Numărul de elemente din arbore

all **:** $\boldsymbol{(Elem \rightarrow Bool) * BinTree \rightarrow Bool}$

Întoarce True dacă p(x) este adevărat pentru toate elementele x din Tree, False altfel

Axiomele pentru All:

(All1): $all(p, empty) = True$

(All2): $all(p, node(left, root, right)) = p(root)$ && $all(p, left)$ && $all(p, right)$

countif **:** $\boldsymbol{(Elem \rightarrow Bool) * BinTree \rightarrow N}$

countif(p, tree) = câte elemente din tree satisfac predicatul p (adică pentru câte elemente din tree p(x) este adevărat

Scrieți axiomele pentru operatorii size și countif, apoi demonstrați prin inducție structurală:

$$\boldsymbol{all(p,t) \rightarrow countif(p,t) == size(t)} \quad (1)$$

AXIOME:

S1: size(empty) = 0

S2: size(node(left, root, right)) = size(left) + size(right) + 1

C1: countif(p, empty) = 0

C2: countif(p, node(left, root, right)) = countif(p, left) + countif(p, right) + (1 if p(root) else 0)

dem: I. caz de bază: empty

? all(p, empty) → countif(p, empty) == size(empty)

All1, C1, S1

(=) true → 0 == 0 ✓

II. inductie

știm: all(p, left) → countif(p, left) == size(left) (I1)

all(p, right) → countif(p, right) == size(right) (I2)

vrem: all(p, node(left, root, right)) → countif(p, ...) == size(...)

dacă: (p(root) && all(p, left) && all(p, right)) All2 =) false → ceva ✓

altfel, true →

C2, S2

(=) countif(p, left) + countif(p, right) + 1 == size(left) + size(right) + 1

der all(p, left) I1 =) ... X

și all(p, right) I2 =) ... Y

=) URA A A A